Diffusion Scores for Markov Data

A technical compendium

Exact inference on the noised chain, estimation of the transition kernel, and comparison with neural denoisers

Giovanni Mantovani
Department of Computing Sciences, Bocconi University
giovanni.mantovani@studbocconi.it

August 17, 2026

Status. Work in progress, circulated internally. This document is the long-form companion to the internal note *Diffusion scores for Markov data*. Where the note states a result, this document derives it. Chapters covering experiments still running are marked [PENDING] and will be completed as those results land; nothing marked pending is relied on elsewhere.

Who this is for. A reader who knows undergraduate probability and linear algebra and nothing about diffusion models, graphical models, or expectation maximisation. Every concept used anywhere in the project is built here from its definition. Nothing is cited in place of an explanation. Where a step is standard but non-obvious, it is done in full rather than asserted, and where an assumption is doing real work, there is a box saying what breaks without it.

Contents

Chapter 1

Setting, conventions, and the three scores

This chapter fixes what the rest of the document assumes. It is deliberately compact: the material here is developed from first principles in the thesis (Hamiltonian and statistical mechanics, stochastic calculus, the diffusion literature), and repeating that development would turn a technical supplement into a second textbook. What is kept is what later chapters actually cite — the estimation-theoretic facts that Proposition 6.1 rests on, the coordinatewise structure of the channel that makes the whole project possible, and the distinction between three different objects that the word "score" is used for.

1.1 Conditioning and the Bayes-optimal estimator

Bayes' rule, in the form used throughout:

$$P(a \mid x) \;=\; \frac{P(x \mid a)\,P(a)}{P(x)} \;\propto\; \underbrace{P(x \mid a)}_{\text{likelihood}}\,\underbrace{P(a)}_{\text{prior}}. \tag{1.1}$$

Proposition 1.1 (Orthogonality of the posterior mean)**.** *For any estimator u that is a function of x alone,*

$$\mathbb{E}\big[\|a-u\|^2 \,\big|\, x\big] \;=\; \mathbb{E}\big[\|a-\langle a\rangle_{x,t}\|^2 \,\big|\, x\big] \;+\; \|u-\langle a\rangle_{x,t}\|^2. \tag{1.2}$$

Proof. Expand $a-u=(a-\langle a\rangle_{x,t})+(\langle a\rangle_{x,t}-u)$. The cross term vanishes because $\langle a\rangle_{x,t}-u$ is x-measurable and $\mathbb{E}[a-\langle a\rangle_{x,t}\mid x]=0$. □

Two consequences are used constantly. The posterior mean minimises the conditional squared risk, so it is the reference every estimator in this project is scored against. And the first term is an irreducible floor common to all estimators, which is why every table reports error *against* $\langle a\rangle_{x,t}$ rather than raw risk — subtracting the floor is what makes the comparison informative.

> Optimality here is with respect to squared error specifically. A different loss selects a different functional of the posterior — absolute error would select the median. Nothing in the project depends on squared loss being the right choice for any application; it is the choice that makes the target computable and the comparison well posed.

1.2 Linear estimators

Restricting to estimators linear in x gives the LMMSE estimator, which for zero-mean (a,x) is

$$\widehat{\mathbf{a}}_{\mathrm{LMMSE}} \;=\; \mathrm{Cov}(a,x)\;\mathrm{Cov}(x)^{-1}x. \tag{1.3}$$

Proposition 1.2 (Linear optimality)**.** *Among estimators of the form $u = Bx$,* (1.3) *minimises $\mathbb{E}\|a - u\|^2$. When (a, x) is jointly Gaussian and zero-mean it coincides with $\langle a\rangle_{x,t}$, since the conditional expectation is then linear.*

The zero-mean hypothesis is load-bearing rather than cosmetic: the general form carries $\mathbb{E}[a] + \mathrm{Cov}(a, x)\,\mathrm{Cov}(x)^{-1}(x - \mathbb{E}[x])$, and dropping the mean terms when $\mathbb{E}[a] \neq 0$ produces a genuinely different estimator. Chapter 6 measures the size of that difference.

1.3 Two things a distribution can be

A model is fitted to samples, not to a distribution. Writing the empirical measure

$$\widehat{P}_0 \;=\; \frac{1}{M}\sum_{\mu=1}^{M} \delta_{a^\mu}, \tag{1.4}$$

the distinction between P_0 and $\widehat{P}_0$ is the one §2.3 turns into the project's motivation. It is not a technicality about estimation error: the two have genuinely different scores, and the reverse process driven by one does something categorically different from the reverse process driven by the other.

1.4 The forward process

The Ornstein–Uhlenbeck channel,

$$\frac{\mathrm{d}x}{\mathrm{d}t} \;=\; -x + \sqrt{2}\,\eta(t), \qquad x(0) = a, \tag{1.5}$$

with η standardised Gaussian white noise, so the $\sqrt{2}$ is carried explicitly. Solving by integrating factor gives

$$x(t) \;=\; e^{-t}a + \sqrt{\Delta_t}\,z, \qquad \Delta_t = 1 - e^{-2t}, \qquad z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})\,, \tag{1.6}$$

and hence the noised distribution as a Gaussian convolution,

$$P_t(x) \;=\; \int \mathrm{d}a\; P_0(a)\, \frac{1}{(2\pi\Delta_t)^{L/2}} \exp\!\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right]. \tag{1.7}$$

Two properties are used repeatedly: $\Delta_t \to 0$ as $t \to 0$, so $P_t \to P_0$; and $\mathrm{Var}(x_t) = 1$ for unit-variance data at every t, so initialising the reverse process at $\mathcal{N}(\mathbf{0}, \mathbf{I})$ incurs a bias of only $e^{-2t_{\max}}$.

1.5 Why the coordinatewise structure matters

This section is short and is the reason the project exists. The forward noise is independent across coordinates, so the likelihood factorises sitewise:

$$P(x \mid a, t) = \prod_{i=1}^{L} \frac{1}{\sqrt{2\pi\Delta_t}} \exp\!\left[-\frac{(x_i - e^{-t}a_i)^2}{2\Delta_t}\right] \;=\; \prod_{i=1}^{L} \ell_t(x_i \mid a_i). \tag{1.8}$$

Assumption at work

Each factor $\ell_t(x_i \mid a_i)$ has degree one among the latent variables. Attaching a degree-one factor to a graph adds a leaf and cannot create a cycle — so conditioning on x reweights

node potentials without creating edges among the a_i, and the posterior factor graph is whatever the prior's was.
Conditioning usually creates dependence. It does not here, precisely because each observation is generated from a single latent variable. The construction fails the moment the forward noise is correlated across coordinates: then $P(x \mid a)$ does not factorise, the likelihood becomes a factor touching several a_i, and the posterior graph acquires edges the prior did not have. Everything in Chapters 5–8 rests on this one property.

1.6 Reversing it

By the time-reversal theorem for diffusions [1, 2], a forward SDE $\mathrm{d}x = f\,\mathrm{d}t + g\,\mathrm{d}W$ has a time-reversed process with drift $f - g^2\nabla \log p_t$ and the same diffusion coefficient. With $f = -x$ and $g = \sqrt{2}$, writing $\tau = t_{\max} - t$ so that increasing τ means decreasing t,

$$\frac{\mathrm{d}x}{\mathrm{d}\tau} \;=\; x + 2\,S(x,\tau) + \sqrt{2}\,\eta(\tau), \qquad S(x,t) := \nabla_x \log P_t(x), \tag{1.9}$$

which is the only object the reverse process needs. Chapter 2 shows it is a rescaling of the posterior mean, and therefore that generating and inferring are the same problem.

1.7 Map of the code

Everything referenced in this document lives under `research/nongaussian-bp/`.

Path	Contents
`src/priors.py`	The data-generating models: `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each provides `sample` and `log_transition_matrix`.
`src/bp_grid.py`	The inference recursion. `make_grid`, `grid_bp` (one sequence, with evidence), `grid_bp_batch` (many sequences, device-parameterised).
`src/bp_gaussian.py`	The closed-form Gaussian recursion in information form.
`src/exact_scores.py`	Closed-form Gaussian posterior mean and score, used as ground truth.
`src/em.py`	The expectation step, the sufficient statistic, and the estimation driver.
`src/kernels.py`	Parametrised transition families and their maximisation steps.
`src/denoiser.py`	A common interface over inference-based and network-based denoisers, plus the network training loop.
`src/reverse.py`	Reverse-time integration: SDE and probability-flow ODE.
`src/sample_metrics.py`	Statistics for comparing generated data against the truth.
`src/backend.py`	CPU/GPU array-module dispatch for the recursion.
`src/ring.py`	The rotating-ring model of Chapter 10: gauge, the closed-form ψ-conditional density, and the blind per-frame likelihood.
`experiments/frozen_config.py`	The single configuration behind every number in the note and the workshop paper. Imported, never re-specified.
`experiments/`	One file per experiment, each self-describing. `exp_28` is the ring.
`tools/check_paper.sh`	The production gate: page limits, no code references, no unfilled placeholders, shared sections still shared.
`tools/check_replicates.py`	Parses each experiment's settings and fails on a replicate count the frozen config cannot reach. See §11.2.
`tests/`	Validation. Several tests here are the evidence for claims made in this document and are cited as such.

Chapter 2

The score is the posterior mean

This chapter proves the identity that makes the whole project possible: the score, which is what the reverse process needs, is a rescaling of the posterior mean, which is what inference computes. They are the same problem.

2.1 The derivation

Start from (1.7) and differentiate with respect to x. The only x-dependence is in the Gaussian factor, and

$$\nabla_x \exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right] = -\frac{x - e^{-t}a}{\Delta_t}\,\exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right],$$

so

$$\nabla_x P_t(x) = \int \mathrm{d}a\; P_0(a)\;\left(-\frac{x - e^{-t}a}{\Delta_t}\right)\frac{1}{(2\pi\Delta_t)^{L/2}}\exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right]. \tag{2.1}$$

Now divide by $P_t(x)$. By Bayes' rule (1.1), the quantity

$$\frac{P_0(a)\,(2\pi\Delta_t)^{-L/2}\exp\!\big[-\|x - e^{-t}a\|^2/2\Delta_t\big]}{P_t(x)}$$

is exactly the posterior density $P(a \mid x, t)$: numerator is prior times likelihood, denominator is the evidence. So (2.1) becomes

$$S(x,t) = \frac{\nabla_x P_t(x)}{P_t(x)} = \int \mathrm{d}a\; P(a \mid x, t)\left(-\frac{x - e^{-t}a}{\Delta_t}\right) = -\frac{x}{\Delta_t} + \frac{e^{-t}}{\Delta_t}\int \mathrm{d}a\; P(a \mid x, t)\, a,$$

and the remaining integral is the posterior mean. Hence

$$\boxed{S(x,t) \;=\; \nabla_x \log P_t(x) \;=\; -\frac{x - e^{-t}\,\langle a\rangle_{x,t}}{\Delta_t}.} \tag{2.2}$$

When is the differentiation legitimate? Interchanging ∇_x and $\int \mathrm{d}a$ requires a dominating function. The integrand's gradient is bounded in absolute value by

$$P_0(a)\,\frac{|x - e^{-t}a|}{\Delta_t}\,\frac{e^{-\|x - e^{-t}a\|^2/2\Delta_t}}{(2\pi\Delta_t)^{L/2}},$$

and for x in a neighbourhood of any fixed point this is dominated by $C(1 + \|a\|)P_0(a)$ for a constant C depending on Δ_t and the neighbourhood, since the Gaussian factor is bounded and decays. That dominating function is integrable exactly when P_0 has a finite first moment.

Dominated convergence then applies. All the priors used here have moments of every order, so the condition is not restrictive.

The formula says something concrete. Rearranged, $\langle a \rangle_{x,t} = e^t(x + \Delta_t\, S(x,t))$: to guess the clean signal, take the observation, step along the score by an amount proportional to how noisy things are, and undo the shrinkage. At small t, Δ_t is small and the correction is tiny — the observation is already almost the answer. At large t, $\Delta_t \to 1$ and the correction does all the work.

Implementation

`src/bp_grid.py` → `score_from_posterior_mean` and `src/denoiser.py` → `score_from_mean` both implement (2.2). Every arm in every comparison converts a posterior mean to a score through one of these, so no arm can gain an advantage from a different convention.

2.2 Consequences worth stating

2.2.1 Estimating the score and denoising are one problem

There is no separate "score model". A denoiser and a score model are the same object in two coordinate systems. This is why diffusion models can be trained by a regression loss on clean signals and then used to drive a differential equation — the training target and the sampling requirement are related by (2.2).

2.2.2 The two error measures differ by a known factor

Applying (2.2) to two different denoisers $\widehat{m}$ and $m^\star$ and subtracting, the x terms cancel:

$$\widehat{S} - S^\star = \frac{e^{-t}}{\Delta_t}\,(\widehat{m} - m^\star)\,, \qquad \text{so} \qquad \big\|\widehat{S} - S^\star\big\| = \frac{e^{-t}}{\Delta_t}\,\big\|\widehat{m} - m^\star\big\|. \tag{2.3}$$

Score error and posterior-mean error are therefore *the same measurement* up to the factor e^{-t}/Δ_t. That factor is not mild: it runs from about 6.0 at $t = 0.08$ to about 0.09 at $t = 2.4$, a spread of roughly 65.

Assumption at work

Reporting only a score error silently weights the low-noise end of the schedule about sixty-five times more heavily than the high-noise end, relative to reporting a posterior-mean error. Two methods can therefore be ranked differently by the two metrics without either metric being wrong. This project reports both, and states the factor, so that a reader can convert. Earlier work here reported score error alone.

Implementation

`src/metrics.py` → `score_mean_identity_residual` checks (2.3) holds to machine precision for any pair of denoisers — a large residual means an implementation bug, not a modelling difference. `src/sample_metrics.py` → `pointwise_ladder` emits `score_reweighting` next to every error it reports. Verified in `tests/test_sample_metrics.py` → `test_score_reweighting_matches_the_tweedie_factor` and `test_reweighting_spans_the_claimed_range_across_the_schedule`.

2.2.3 The Gaussian case, as a check

If $P_0 = \mathcal{N}(0, \Sigma_0)$ then x is Gaussian with covariance $\Sigma_t = e^{-2t}\Sigma_0 + \Delta_t I$, and the posterior mean is linear:

$$\langle a \rangle_{x,t} = e^{-t}\Sigma_0\Sigma_t^{-1}x, \qquad S(x,t) = -\Sigma_t^{-1}x. \tag{2.4}$$

The second follows by differentiating $\log P_t(x) = -\frac{1}{2}x^\top \Sigma_t^{-1} x + \text{const}$ directly, and consistency with (2.2) is a one-line check that we use as a test of the whole pipeline.

Implementation

`src/exact_scores.py` → `exact_gaussian_posterior_mean` and `src/exact_scores.py` → `precision_matrix_score` implement (2.4). These are the ground truth against which the general machinery of Chapter 5 is validated; see `tests/test_score_mean_error_identity.py`.

2.3 Three scores, not one

Equation (2.2) defines the score of *a* distribution. Which distribution one has in mind changes what the object is, and three different choices appear in the literature under the same word.

The population score. Take P_0 to be the true data distribution. This is the score that the reverse process would need in order to generate genuinely new data distributed as P_0. Nobody has it, because nobody has P_0.

The empirical score. Take $\widehat{P}_0$ from (1.4), the sum of point masses at the training examples. Its noised version is a Gaussian mixture centred on those examples, and its posterior mean is a softmax-weighted average of them:

$$\widehat{m}(x,t) = \frac{\sum_\mu a^\mu \exp\bigl[-\|x - e^{-t}a^\mu\|^2/2\Delta_t\bigr]}{\sum_\nu \exp[-\|x - e^{-t}a^\nu\|^2/2\Delta_t]}. \tag{2.5}$$

Note what this returns as $t \to 0$: the nearest training example. Exact reversal under this score reproduces the training set and nothing else.

The learned score. What a network with finite capacity, finite data and a particular optimiser actually produces.

Assumption at work

The chain of reasoning that matters, stated carefully:

1. Training minimises an average over the observed samples — the *empirical* objective.
2. Its unrestricted minimiser over all measurable functions is the *empirical* posterior mean (2.5), by Proposition 1.1 applied under $\widehat{P}_0$.
3. Exact reversal under that score returns $\widehat{P}_0$ — the training set.
4. Therefore a model that perfectly attained the optimum of its own training objective would be useless as a generative model.

That trained models nonetheless generalise is a statement about what stops them from attaining that optimum — capacity, architecture, optimisation, regularisation — not about the objective. It is *not* correct to say the population objective is minimised by the empirical score; the population objective is minimised by the population posterior mean.

This is the tension that motivates wanting a setting where the *population* score is computable. If one can compute it, one can ask how far a trained network is from it, and separate "the network is bad" from "this target is hard". That is what the Markov chain of Chapter 3 provides.

Sources

The identity between the score of a Gaussian-convolved density and the posterior mean is classical in empirical Bayes, due to Robbins [3] and Miyasawa [4], and is usually called Tweedie's formula after the exposition in Efron [5]. Its use as a training objective for generative models is Vincent [6], building on the score-matching estimator of Hyvärinen [7]; the generative-modelling application is Song and Ermon [8].

Chapter 3

The data model: a Markov chain

3.1 Definition

A sequence $a = (a_1, \dots, a_L)$ is a *first-order Markov chain* if, given the present, the future is independent of the past:

$$P(a_i \mid a_{i-1}, a_{i-2}, \dots, a_1) \;=\; P(a_i \mid a_{i-1}) \qquad \text{for all } i. \tag{3.1}$$

Applying the chain rule of probability and then (3.1) repeatedly,

$$\boxed{P_0(a) \;=\; \mu(a_1) \prod_{i=2}^{L} K(a_i \mid a_{i-1}),} \tag{3.2}$$

where μ is the law of the first site and K is the *transition kernel*. We take K homogeneous — the same rule at every step — which will matter in Chapter 5 and again when we estimate it.

The factorisation is the whole point. A general density on $\mathbb{R}^L$ is an unmanageable object. Equation (3.2) says this one is built from $L-1$ copies of a single two-variable function. Everything tractable about the model descends from that.

3.2 The specific family used here

We use additive transitions with a linear autoregression,

$$a_i \;=\; \rho\, a_{i-1} + \varepsilon_i, \qquad \varepsilon_i \sim \varphi \text{ i.i.d.}, \qquad K(a' \mid a) = \varphi(a' - \rho a), \tag{3.3}$$

with $|\rho| < 1$, and φ an *innovation density* of mean zero and variance q.

3.2.1 Covariance stationarity

Suppose a_1 is drawn with $\mathrm{Var}(a_1) = \sigma^2$ and ask when that variance is preserved along the chain. Taking variances in (3.3) and using independence of ε_i from a_{i-1},

$$\sigma^2 = \rho^2 \sigma^2 + q \qquad \Longrightarrow \qquad \sigma^2 = \frac{q}{1-\rho^2}.$$

Choosing $q = 1 - \rho^2$ therefore gives unit marginal variance. For the covariance, multiply (3.3) by a_{i-k} and take expectations; since ε_i is independent of everything earlier,

$$\mathrm{Cov}(a_i, a_{i-k}) = \rho\, \mathrm{Cov}(a_{i-1}, a_{i-k}) = \cdots = \rho^k \sigma^2,$$

so with the normalisation above

$$\boxed{(\Sigma_0)_{ij} \;=\; \mathrm{Cov}(a_i, a_j) \;=\; \rho^{|i-j|}.} \tag{3.4}$$

Correlation decays geometrically with separation, with ρ setting the range. At $\rho = 0.85$, sites five apart still share $\rho^5 \approx 0.44$ of their variation; sites twenty apart share 0.04. There is a well-defined correlation length, and it will reappear in Chapter 6 as the scale controlling how much context a denoiser needs.

3.2.2 The crucial degree of freedom

Here is the feature that makes this family a good experimental probe, and it is worth dwelling on.

Equation (3.4) depends on φ *only through its variance* q. Everything else about the innovation — whether it is Gaussian, heavy-tailed, bounded, bimodal — leaves no trace in the covariance whatsoever.

So one can build a family of data distributions that are

- *identical* in every second-order statistic, and
- *different* in every higher-order one.

family	innovation	excess kurtosis	covariance
`GaussianAR1`	$\mathcal{N}(0, q)$	0	$\rho^{\lvert i-j\rvert}$
`LaplaceAR1`	Laplace$(0, b)$, $2b^2 = q$	$+3$	$\rho^{\lvert i-j\rvert}$
`StudentTAR1`	scaled t_ν	$6/(\nu-4)$	$\rho^{\lvert i-j\rvert}$
`UniformAR1`	uniform on $[-c, c]$	-1.2	$\rho^{\lvert i-j\rvert}$
`GaussianMixtureAR1`	symmetric two-component	< 0, bimodal	$\rho^{\lvert i-j\rvert}$

This is the experimental design in one table. Any method that uses only second-order information — and Chapter 6 shows that a very standard approximation does exactly that — cannot tell these five apart, by construction. Any difference in performance across the row is therefore attributable to higher-order structure and nothing else. There is no confound to argue about.

Implementation

`src/priors.py` → `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each is a frozen dataclass holding only ρ, with `q`, `sample`, `log_transition_matrix` and `innovation_excess_kurtosis` as properties. The variance matching of §3.2.2 is built into each class rather than imposed by the caller, so the families cannot accidentally be compared at mismatched covariance.

3.3 Why this is a reasonable model of anything

Two defences, and it is worth being honest that they are defences rather than derivations.

Sequential data has temporal coherence. Diffusion models are increasingly applied to video and other sequential data, where consecutive frames are strongly dependent. A first-order Markov chain is the simplest non-trivial model of that dependence. It is not a model *of* video; it is a model of the property that makes video different from a bag of independent images.

It is a controllable probe. The stronger justification is methodological. The set of distributions for which the population score is computable is small, and every member of it is a modelling compromise. This one buys genuine nonlinearity of the score — which Gaussians do not have — at the price of a strong structural assumption. Given that the aim is to study how estimators exploit structure, having structure one can dial is the point.

Assumption at work

The model assumes *exact* first-order Markov structure. If the data has longer-range dependence, an estimator built on (3.2) is misspecified, and no amount of data repairs a misspecification. This is the single largest limitation of everything that follows, and it is not mitigated anywhere in this document.

3.4 Sampling from the chain

Generating data is direct: draw a_1 from μ, then iterate (3.3). There is no rejection, no burn-in, no approximation.

Implementation

Each prior's `sample(rng, n)` returns *one* sequence of length `n`, built by the recursion above with a_1 drawn standard normal. Batches are formed by stacking, e.g. `experiments/exp_16_sampling_validation.py` → `sample_chains`. The single-sequence signature is a deliberate simplicity, not an oversight; batching is the caller's business.

Remark 3.1 (Covariance-stationary, not strictly stationary). The computation above establishes exactly one thing: with $q = 1 - \rho^2$ and $\mathrm{Var}(a_1) = 1$, *second* moments propagate unchanged, so $\mathrm{Var}(a_i) = 1$ for every i and $\mathrm{Cov}(a_i, a_j) = \rho^{|i-j|}$, whatever the shape of φ. The chain is **covariance-stationary** (equivalently second-order stationary).

It is **not** strictly stationary in distribution unless φ is Gaussian. The invariant law of $a_i = \rho a_{i-1} + \varepsilon_i$ is the law of $\sum_{k\geq 0} \rho^k \varepsilon_{-k}$, an infinite convolution which for non-Gaussian ε is not Gaussian; drawing $a_1 \sim \mathcal{N}(0, 1)$ therefore starts the chain off its invariant law, and the marginal of a_i drifts towards that law over a burn-in of order $1/\log(1/\rho) \approx 6$ sites at $\rho = 0.85$. Choosing $q = 1 - \rho^2$ alone does not buy strict stationarity, and this document does not claim it does.

Nothing in the comparisons depends on the difference. Every family shares the same a_1 law and the same covariance, so they still differ only beyond second moments, which is the property the controlled probe needs. But the alternative — sampling a_1 from each family's invariant law, which needs a fixed-point computation per family — has not been run, so the claim that it would change nothing is a prediction and not a measurement. It is on the list in §9.7.

Chapter 4

Graphical models and factor graphs

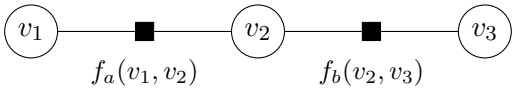
Chapter 3 gave a factorised density. This chapter turns factorisation into a picture, because the picture is what makes the key result of Chapter 5 obvious rather than surprising.

4.1 Factor graphs

A *factor graph* represents a density that is a product of local terms,

$$P(v_1, \ldots, v_N) \;\propto\; \prod_{\alpha} f_\alpha(v_{\partial\alpha})\,, \tag{4.1}$$

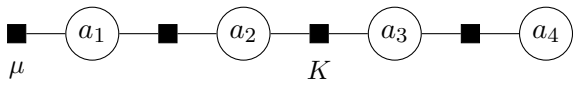
where each factor f_α depends on a subset $\partial\alpha$ of the variables. The graph is bipartite: a circle for each variable, a square for each factor, and an edge between them whenever the variable appears in that factor.



The graph records *which variables interact directly.* Two variables not sharing a factor influence each other only through intermediaries. That is the entire content of the representation, and it is what will let us reorganise an intractable integral.

4.2 The Markov chain as a factor graph

For (3.2), the factors are $\mu(a_1)$ and one $K(a_i \mid a_{i-1})$ per consecutive pair. Each transition factor touches exactly two variables, adjacent ones, so the graph is a path:



4.3 Trees, and why they are special

Definition 4.1. A factor graph is a *tree* if it contains no cycles: between any two nodes there is exactly one path.

A path graph is a tree. This is the structural fact everything rests on.
The reason trees are special is the *separator* property.

Proposition 4.2 (Separation)**.** *Let the graph be a tree and let a_i be a variable node. Removing a_i disconnects the graph into components. Conditional on a_i, the variables in different components are independent.*

Proof. Because the graph is a tree, every factor involves variables lying in $\{a_i\}$ together with exactly one component — if a factor touched two components it would create a second path between them, hence a cycle. Group the factors by component: $P \propto \prod_c g_c\big(a_i, v^{(c)}\big)$ where $v^{(c)}$ are the variables of component c. Fixing a_i, the density in the remaining variables factorises across components, which is independence. □

On a chain, this says: if you know a_i, then knowing anything to its left tells you nothing further about anything to its right. All the influence between the two sides is routed through a_i. That is why one can summarise everything to the left of site i in a single function of a_i — there is nothing else about the left that the right needs to know.

Assumption at work

The proof used acyclicity twice. On a graph with a cycle, a factor *can* join two components, the density does not factorise on conditioning, and the summary function is no longer sufficient. Belief propagation applied anyway ("loopy BP") double-counts information travelling around the cycle: a message leaving a node returns to it having been treated as independent evidence. It may still work well in practice, but it is no longer exact, and none of the guarantees in Chapter 5 survive.

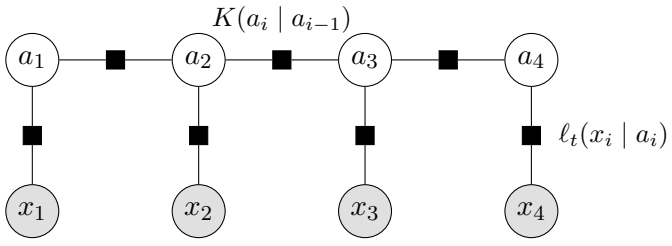
4.4 Conditioning on the observations

Now the step that makes the project work.

We observe x, related to a by (1.6). By Bayes' rule the posterior is prior times likelihood, and by (1.8) the likelihood factorises over sites:

$$P(a \mid x, t) \;\propto\; \underbrace{\mu(a_1) \prod_{i=2}^{L} K(a_i \mid a_{i-1})}_{\text{prior: chain}} \cdot \underbrace{\prod_{i=1}^{L} \ell_t(x_i \mid a_i)}_{\text{likelihood: unary}} . \tag{4.2}$$

Every likelihood factor touches *one* latent variable. In factor-graph terms it is a pendant node hanging off a_i and connected to nothing else.



Proposition 4.3. *The factor graph of the posterior* (4.2) *is a tree.*

Proof. The latent variables and transition factors form a path, which is acyclic. Each likelihood factor has degree one among latent variables, so attaching it adds a leaf and cannot create a cycle. The observations are fixed, not variables. □

This is the whole trick, and it is worth saying plainly why it is not obvious. Conditioning usually *creates* dependence — explaining away is the standard example. Here it does not, because each observation is generated from a single latent variable independently of all others. So conditioning reweights the node potentials and leaves the edge set alone. The posterior is a different chain, not a non-chain.

Assumption at work

This fails the moment the noise is correlated across coordinates. Then $P(x \mid a)$ does not factorise, the likelihood becomes a factor touching several a_i simultaneously, and the posterior graph acquires edges — generally becoming dense and losing tractability entirely. The coordinatewise structure of the OU noise (§1.5) is doing essential work here.

4.5 The cost that has been removed

Marginalising (4.2) naively to get $P(a_i \mid x, t)$ means integrating over $L - 1$ variables — exponential in the sequence length if done by brute force on a discretisation. Proposition 4.3 says the posterior has the structure that makes this cost linear instead. Chapter 5 carries that out.

Sources

Factor graphs and the sum-product algorithm in the form used here are Kschischang et al. [9]; the graphical-model formulation and the separation properties are Pearl [10]. For the statistical-physics reading of the same objects, including exactness on trees, see Mézard and Montanari [11]; for the exponential-family and variational view, Wainwright and Jordan [12].

Chapter 5

Belief propagation

Chapter 4 showed the posterior is a tree. This chapter turns that into an algorithm and proves it exact.

5.1 The problem

We want $P(a_i \mid x, t)$ for each i, from

$$P(a \mid x, t) \;\propto\; \mu(a_1) \prod_{i=2}^{L} K(a_i \mid a_{i-1}) \prod_{i=1}^{L} \ell_t(x_i \mid a_i). \tag{5.1}$$

By definition this means integrating out the other $L-1$ variables. Discretising each variable to N_g values and summing naively costs N_g^{L-1} operations — at $N_\mathrm{g} = 401$, $L = 32$ that is about 10^{80}, so the naive route is not merely slow but impossible.

5.2 The idea: factor out what does not depend on the summation variable

Consider a small case, $L = 3$, and write $\psi_i(a_i) := \ell_t(x_i \mid a_i)$. We want $P(a_3 \mid x)$, so we integrate over a_1 and a_2:

$$P(a_3 \mid x) \propto \psi_3(a_3) \int \mathrm{d}a_2 \int \mathrm{d}a_1 \; \mu(a_1)\psi_1(a_1)\, K(a_2 \mid a_1)\, \psi_2(a_2)\, K(a_3 \mid a_2).$$

The inner integral over a_1 involves only the first three factors — $K(a_3 \mid a_2)$ does not contain a_1, so it can be pulled out. Define

$$\overrightarrow{m}_2(a_2) := \int \mathrm{d}a_1 \; \mu(a_1)\, \psi_1(a_1)\, K(a_2 \mid a_1),$$

a function of a_2 alone. Then

$$P(a_3 \mid x) \propto \psi_3(a_3) \int \mathrm{d}a_2 \; \overrightarrow{m}_2(a_2)\, \psi_2(a_2)\, K(a_3 \mid a_2) =: \psi_3(a_3)\, \overrightarrow{m}_3(a_3).$$

Two integrals became two *nested* integrals, each over one variable. The saving is the whole algorithm: instead of a single sum over a grid of size N_g^2, we do two sums of size N_g each, reusing the first result. That is dynamic programming, and it works because the factorisation lets the summand split.

5.3 The general recursion

Generalising, define the *forward* (left) and *backward* (right) messages

$$\boxed{\begin{aligned} \overrightarrow{m}_i(a_i) \;&\propto\; \int \mathrm{d}a_{i-1}\; K(a_i \mid a_{i-1})\, \psi_{i-1}(a_{i-1})\, \overrightarrow{m}_{i-1}(a_{i-1}), && \overrightarrow{m}_1 := \mu, \qquad (5.2)\\ \overleftarrow{m}_i(a_i) \;&\propto\; \int \mathrm{d}a_{i+1}\; K(a_{i+1} \mid a_i)\, \psi_{i+1}(a_{i+1})\, \overleftarrow{m}_{i+1}(a_{i+1}), && \overleftarrow{m}_L :\equiv 1. \qquad (5.3) \end{aligned}}$$

The base cases matter. $\overrightarrow{m}_1 = \mu$ is the only place the initial law enters; without it the prior on the first site would be silently dropped. $\overleftarrow{m}_L \equiv 1$ says there is no evidence to the right of the last site — an improper flat function, which is fine because it is always multiplied by something integrable.

Proposition 5.1 (Exactness)**.** *With* (5.2)–(5.3)*,*

$$P(a_i \mid x, t) \;\propto\; \overrightarrow{m}_i(a_i)\, \psi_i(a_i)\, \overleftarrow{m}_i(a_i), \tag{5.4}$$

and the pairwise marginal on the edge $(i-1, i)$ *is*

$$P(a_{i-1}, a_i \mid x, t) \;\propto\; \overrightarrow{m}_{i-1}(a_{i-1})\, \psi_{i-1}(a_{i-1})\; K(a_i \mid a_{i-1})\; \psi_i(a_i)\, \overleftarrow{m}_i(a_i). \tag{5.5}$$

Proof. By induction. Unrolling (5.2),

$$\overrightarrow{m}_i(a_i) \propto \int \mathrm{d}a_{1:i-1}\; \mu(a_1) \prod_{j=2}^{i} K(a_j \mid a_{j-1}) \prod_{j=1}^{i-1} \psi_j(a_j),$$

which is exactly the joint density of everything at or left of site i, with the left variables integrated out and ψ_i *not* yet included. Symmetrically $\overleftarrow{m}_i(a_i)$ is the joint over everything strictly right of i, integrated out, given a_i. By Proposition 4.2 these two are conditionally independent given a_i, so their product times the local evidence $\psi_i(a_i)$ is proportional to the marginal. The pairwise statement is the same argument with the separator taken to be the edge rather than a node. □

Assumption at work

Note where ψ_i appears. In (5.2) the local evidence at site $i-1$ is absorbed *before* the transition, so $\overrightarrow{m}_i$ carries $\psi_1, \ldots, \psi_{i-1}$ and $\overleftarrow{m}_i$ carries $\psi_{i+1}, \ldots, \psi_L$. The marginal (5.4) therefore contains each ψ_j exactly once. Absorbing the local evidence *after* the transition instead would put ψ_i into both messages and square it — the belief would be sharper than the truth, and every posterior variance too small. Getting this wrong produces plausible-looking output, which is why the convention is pinned by a test rather than left to a comment.

Implementation

`src/bp_grid.py` → `grid_bp` implements (5.2)–(5.4) for one sequence, returning the beliefs and the log-evidence. `src/bp_grid.py` → `grid_bp_batch` does the same for a batch, sharing the site loop so each message update is a single $(N_\mathrm{g} \times N_\mathrm{g}) \cdot (N_\mathrm{g} \times B)$ matrix product. The initialisations are literally `L[0] = exp(log_mu)` and `R[-1] = 1.0`. Pairwise statistics (5.5) are accumulated in `src/em.py` → `e_step`.

5.4 Cost

Each message update integrates one variable against the kernel: on a grid of N_g points that is a matrix–vector product, $O(N_g^2)$. There are $2L$ updates, so one sequence costs

$$O(LN_g^2) \tag{5.6}$$

instead of $O(N_g^L)$. At $L = 32$, $N_g = 401$ this is about 5×10^6 operations — microseconds — against the 10^{80} of the naive route.

> The exponential became linear in the sequence length. Nothing was approximated to achieve it; the saving is entirely from reorganising the order of integration, which the tree structure permits and a general graph does not.

5.5 Reading off the answer

From the belief $b_i(a_i) := P(a_i \mid x, t)$ we get the posterior mean by one more integral,

$$\langle a_i \rangle_{x,t} = \int \mathrm{d}a_i \; a_i \, b_i(a_i), \tag{5.7}$$

and then the score from (2.2). That is the complete pipeline: messages, beliefs, mean, score.

Implementation

`src/denoiser.py` → `bp_posterior_mean` is the single entry point used by every experiment — it takes anything exposing `log_transition_matrix`, which covers both the true priors of `src/priors.py` and the estimated kernels of `src/kernels.py`. That shared interface is what makes "exact inference under the true kernel" and "inference under a learned kernel" the same code path with a different argument, so no comparison between them can be confounded by an implementation difference.

5.6 Evidence, and why it is worth carrying

The normalising constant of (5.4) is the *evidence* $P_t(x)$. Messages are renormalised at each step to prevent underflow, and if the discarded constants are accumulated, their product recovers $\log P_t(x)$ exactly.

This is not bookkeeping for its own sake. The evidence is the objective that the estimation of Chapter 8 maximises, and having it in closed form is what makes it possible to check a gradient against a finite difference of the thing being optimised.

Implementation

`src/bp_grid.py` → `grid_bp` returns `log_evidence` alongside the beliefs. `src/exact_scores.py` → `gaussian_log_evidence` gives the closed form in the Gaussian case, which is what the general routine is tested against.

5.7 The relationship to forward–backward and to Kalman

Equations (5.2)–(5.3) are the forward–backward algorithm of hidden Markov models, written for a continuous state. If the latent state is discrete the integrals become sums and this is exactly Baum's algorithm; if everything is Gaussian, messages are determined by two numbers each and the recursion becomes the Kalman filter and RTS smoother.

These are not three algorithms but one, specialised by what the messages are. Discrete: a vector. Gaussian: a mean and a variance. General continuous: a function, which must be represented somehow — and *that* representation is the only approximation in the whole scheme. Chapter 7 is about it.

5.8 Validation

Two independent checks establish that the implementation computes what this chapter claims.

Against brute force. For a short chain and a coarse grid, the discretised posterior can be enumerated exhaustively — all N_{g}^{L} configurations — and marginalised by summation, using no messages at all. This shares no code with the recursion. Agreement is at machine precision: posterior means to 1.6×10^{-14}, log-evidence to 1.8×10^{-15}, and the pairwise statistic to 3.9×10^{-16}, with the total pairwise mass equal to the edge count exactly.

Against a closed form. On a Gaussian chain the posterior mean is known analytically (2.4), and the recursion reproduces it to 9.2×10^{-15} relative at the working resolution.

Implementation

`tests/test_em_bp.py` contains the enumeration check; `tests/test_grid_convergence.py` and `tests/test_score_mean_error_identity.py` contain the Gaussian and identity checks. These tests are the evidence for the numbers above and are the reason those numbers can be quoted at all.

Sources

The recursions of this chapter are the forward–backward algorithm for hidden Markov models [13], of which Rabiner [14] is the standard exposition. In the linear-Gaussian case they specialise to the Kalman filter [15] and the Rauch–Tung–Striebel smoother [16]. Cappé et al. [17] treats the continuous-state case in the generality needed here. That U-Net architectures can be read as approximate belief propagation on a hierarchical model is argued by Mei [18].

Chapter 6

The Gaussian case and the LMMSE baseline

The Gaussian chain plays two roles: it is the case where everything is computable in closed form, hence the yardstick for validating the general machinery; and it is the model implicitly assumed by the most common approximation, hence the baseline everything is measured against.

6.1 Everything in closed form

Take $\varphi = \mathcal{N}(0, q)$. Then a is jointly Gaussian with covariance $\Sigma_0 = \rho^{|i-j|}$ from (3.4), and by (1.6) so is x, with

$$\Sigma_t \;=\; e^{-2t}\Sigma_0 + \Delta_t I. \tag{6.1}$$

For jointly Gaussian variables the conditional mean is linear, giving (2.4): $\langle a\rangle_{x,t} = e^{-t}\Sigma_0\Sigma_t^{-1}x$ and $S = -\Sigma_t^{-1}x$.

> The score is a fixed linear map. No inference is required at all — one matrix solve. This is exactly why Gaussian data is a poor probe of how models exploit structure: the answer is linear regression, and every method that can do linear regression is optimal.

6.1.1 The precision matrix is tridiagonal

A useful structural fact. The clean precision $Q_0 = \Sigma_0^{-1}$ of an AR(1) chain is *tridiagonal*: writing the density as $P_0(a) \propto \exp[-\frac{1}{2q}\sum_i (a_i - \rho a_{i-1})^2]$ and expanding, only terms a_i^2 and $a_i a_{i-1}$ appear. So

$$(Q_0)_{ii} = \frac{1+\rho^2}{q} \text{ (interior)}, \qquad (Q_0)_{i,i\pm1} = -\frac{\rho}{q}, \qquad (Q_0)_{ij} = 0 \text{ for } |i-j| > 1.$$

The posterior precision adds the likelihood's contribution, $J = (e^{-2t}/\Delta_t)I + Q_0$, and stays tridiagonal at every t.

> Tridiagonal precision is the algebraic image of the Markov property: zero in the precision matrix means conditional independence given everything else. The *covariance* Σ_t is dense — every site is correlated with every other — while the precision is banded. Being Markov is a statement about the precision, not the covariance.

6.2 The moment closure

For a non-Gaussian innovation the messages of Chapter 5 are genuinely non-Gaussian functions. A natural approximation keeps only two numbers per message: replace each message by the

Gaussian with the same mean and variance. This is cheap, requires no grid, and is what one would write first.

Under the transition $a_i = \rho a_{i-1} + \varepsilon_i$ with ε of mean 0 and variance q, a message with moments (m, v) maps to

$$\text{forward:} \quad (m, v) \mapsto (\rho m,\ \rho^2 v + q); \qquad \text{backward:} \quad (m, v) \mapsto (m/\rho,\ (v + q)/\rho^2), \tag{6.2}$$

the backward map being the forward one inverted, because that recursion integrates over the *output* of the transition rather than its input. It requires $\rho \neq 0$.

Proposition 6.1. *For the chain* (3.3) *with* $\mathbb{E}[a] = 0$ *and Gaussian unary likelihoods, the moment-closed recursion returns*

$$\widehat{\mathbf{a}}_{\mathrm{LMMSE}} \;=\; e^{-t}\Sigma_0 \left(e^{-2t}\Sigma_0 + \Delta_t I\right)^{-1} x, \tag{6.3}$$

the LMMSE estimator of the covariance-matched Gaussian model, for every *innovation law with those two moments.*

Proof. Both maps in (6.2) depend on φ only through $(0, q)$, and the likelihood absorption is a Gaussian–Gaussian product, which is exact. So the recursion returns the same function of x for every innovation law with those moments — in particular the same as on the covariance-matched *Gaussian* chain. On that chain every message is genuinely Gaussian, so the projection is the identity, the recursion is exact, and it returns the true posterior mean. For jointly Gaussian zero-mean (a, x) the posterior mean is linear and coincides with the LMMSE estimator of Proposition 1.2. □

Assumption at work

The zero-mean hypothesis is load-bearing. The general LMMSE estimator is $\mathbb{E}[a] + \mathrm{Cov}(a, x)\,\mathrm{Cov}(x)^{-1}(x - \mathbb{E}[x])$, and dropping the mean terms is only valid when both means vanish. With a prior mean of 1.3, (6.3) departs from the true LMMSE by 0.65 in maximum absolute value.

This is a sharper statement than "the Gaussian approximation is approximate". The closure does not merely lose some information about the innovation — it loses *all* of it beyond the variance. Two chains from the matched family of §3.2.2, one Gaussian and one heavily bimodal, are given identical treatment. The approximation is blind by construction, not by degree.

A consequence worth stating: at the single-Gaussian level, "error from representing messages badly" and "error from assuming the wrong model" are the same number. They are different objects only for richer message families.

Implementation

`src/bp_gaussian.py` → `gaussian_chain_bp` implements the closure in information form, carrying precision λ and information h rather than mean and variance. That parametrisation matters: a message conveying no information is $\lambda = 0$ exactly, whereas in moment form it is a variance of $+\infty$.

Assumption at work

An earlier version of this project implemented the closure by evaluating each Gaussian message *on the grid*, pushing it through the exact update, and re-fitting moments. At weakly informative t the outgoing message is nearly flat, and moment-matching a flat function *truncated* to $[-A, A]$ returns a variance of $(2A)^2/12$ rather than ∞ — an invented precision of order $3/A^2$. The error fed back through the recursion, driving messages towards the boundary. Measured maximum absolute error in the posterior mean at $t = 1.8$: $9.1 \times$

10^{-1}, against 4.4×10^{-16} for the information form. The lesson is that the closure must be done analytically; representing an unnormalisable object on a finite grid is not a neutral implementation detail.

6.3 Where the approximation costs something

Since the closure is exactly LMMSE, the gap to exact inference is the value of the higher-order structure. Measured on a Laplace chain at $\rho = 0.85$, the relative deviation of the closure's score from the exact one falls monotonically from 0.198 at $t = 0.08$ to 9.9×10^{-5} at $t = 2.4$.

The direction may be surprising: the approximation is *worst at low noise*. The reason is that at large t the likelihood is weak and the posterior is dominated by the smoothing effect of the noise, which Gaussianises everything — so a Gaussian approximation of a nearly Gaussian object is nearly exact. At small t the posterior is sharply peaked and its shape is inherited from the prior, whose non-Gaussianity is then fully visible.

Whether that matters for *generation* is a separate question, because errors at small t enter the reverse process where the denoiser is barely acting. That question is what the experiments of Chapter 9 address, and the answer is not the obvious one.

Implementation

`experiments/exp_02_laplace_gaussian_message_error.py` → produces the numbers above; `experiments/exp_03_nongaussian_innovation_sweep.py` → extends them across the innovation families of §3.2.2.

Sources

The linear-minimum-mean-square-error estimator and its orthogonality characterisation are standard [19, 20]. The Gaussian message recursions coincide with Kalman filtering and RTS smoothing [15, 16].

Chapter 7

Representing messages: the numerical layer

Chapter 5 gave an exact algorithm on functions. A computer cannot store a function, so an implementation must choose a representation — and that choice is the *only* approximation in the whole scheme. This chapter is about it, because conflating this approximation with the exactness of the previous chapter is the easiest way to overstate what the method does.

7.1 Three statements that must be kept apart

1. **Sum-product on the posterior chain is exact**, as a theorem about tree-structured factorisations (Proposition 5.1). This is a mathematical statement about functional messages and involves no computer.
2. **The discretised recursion is exact for the discretised model.** Restricting messages to a grid defines a different, finite-state model; on that model the recursion is exact up to floating-point arithmetic.
3. **The discretised model approximates the continuous one**, with an error that must be *measured*.

Assumption at work

Writing simply "our method is exact" collapses (1) and (3) and claims something false. Throughout this project we write *exact tree inference* for (1) and *exact for the discretised model* for (2), and never abbreviate. The distinction is not pedantry: (1) holds for any innovation law whatsoever, while the error in (3) depends strongly on the smoothness of that law — as §7.3.2 shows.

7.2 The grid

Messages are represented by their values on N_{g} equally spaced points on $[-A, A]$, and integrals become composite trapezoidal sums,

$$\int \mathrm{d}a\, f(a) \;\approx\; \sum_{k=1}^{N_{\mathrm{g}}} w_k f(u_k), \qquad w_k = \Delta u \text{ (interior)}, \quad \tfrac{1}{2}\Delta u \text{ (endpoints)}. \tag{7.1}$$

Each message update (5.2) becomes a matrix–vector product with the matrix $K(u_l \mid u_k)$, which is formed once per model and reused at every site and every sequence.

Implementation

`src/bp_grid.py` → `make_grid` builds the points and trapezoidal weights. The transition matrix comes from each prior's `log_transition_matrix(grid)` (`src/priors.py`) or each estimated kernel's (`src/kernels.py`) — the same interface, which is why the true and learned models are interchangeable in every experiment.

7.3 Two error sources

7.3.1 Truncation

Probability mass outside $[-A, A]$ is discarded rather than transported. The diagnostic is the mass in the boundary cells: if it is not negligible, A is too small and refining N_g will not help, because the error is in the domain rather than the resolution.

This diagnostic used to be computed inside the recursion and thrown away, so no aggregate could be quoted. It is now recorded (`outputs/exp_18/boundary.csv`, `experiments/exp_18_revision_diagnostics`). Taking the worst case over all 32 sites of a forward sweep on a Laplace chain at $\rho = 0.85$:

A	4	6	8	10
worst boundary mass	3.4×10^{-4}	2.0×10^{-6}	1.4×10^{-8}	1.4×10^{-10}

The value is essentially independent of N_g across $\{201, 401, 801\}$, which is the point: truncation responds to the domain and not to the resolution, and a grid-refinement sweep at fixed A therefore cannot certify it. At the working $A = 8$ the discarded mass is eight orders of magnitude below any effect this project reports.

Assumption at work

This failure is invisible unless one looks for it. An earlier package in this project ran at $A = 8$ with a boundary mass of 7.4×10^{-7}, above its own tolerance, and the symptom was not an error message but a slow drift in the posterior means at large t. Widening the domain at preserved spacing reduced it to 5.3×10^{-10}.

7.3.2 Quadrature

The trapezoidal rule has error $O(\Delta u^2)$ for integrands with a bounded second derivative. Whether that bound applies depends on the innovation.

- **Gaussian innovation.** The integrand is smooth and convergence is fast.
- **Laplace innovation.** $\varphi(e) \propto e^{-|e|/b}$ has a discontinuous derivative at $e = 0$. The second derivative is not bounded there, the standard estimate does not apply, and convergence is second order at best.

Assumption at work

It is tempting — and this project did it in an earlier draft — to describe the discretisation as "spectrally accurate" on the strength of the Gaussian measurements. That generalisation is not available: spectral accuracy requires smoothness the Laplace kernel does not have. The accompanying note now claims second-order behaviour for that family and nothing more.

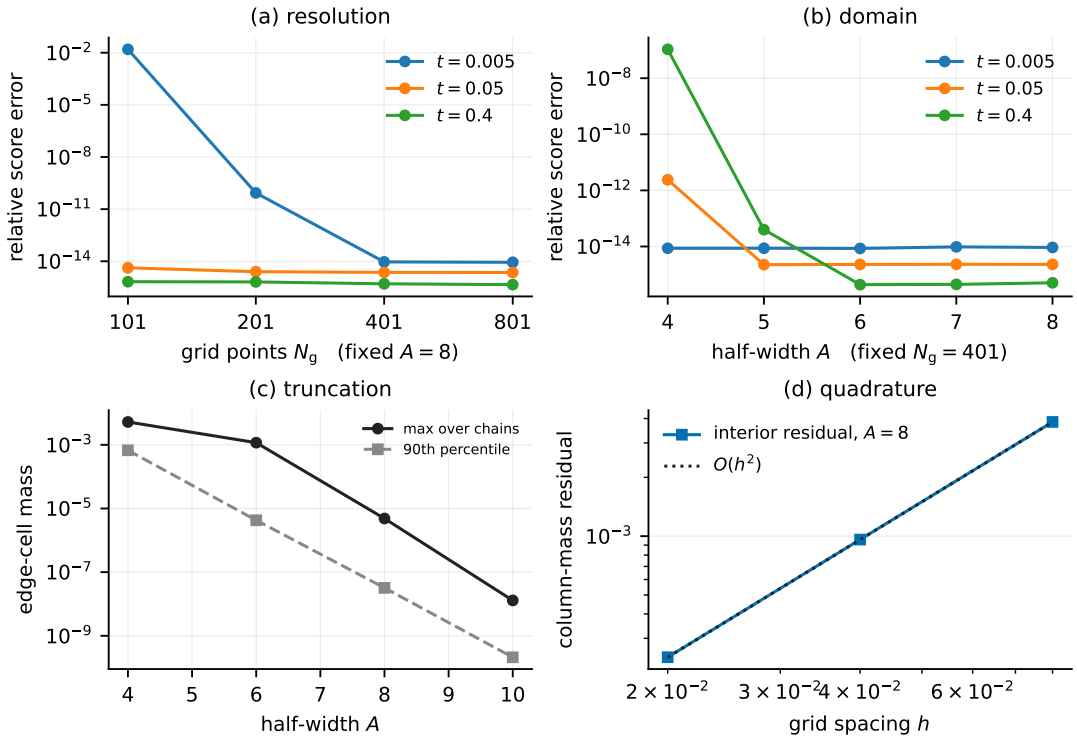


Figure 7.1: The two error sources, measured separately. (a) Relative score error against the number of grid points at fixed $A = 8$, and (b) against the half-width at fixed $N_g = 401$; both saturate at the floating-point floor $\sim 10^{-14}$, beyond which the curves measure arithmetic rather than discretisation. (c) Worst boundary mass over 32 sites against A: the truncation diagnostic, near-independent of N_g. (d) Interior column-mass residual of the transition matrix against spacing at $A = 8$: the quadrature diagnostic, following $O(h^2)$ as the trapezoidal rule predicts. Panels (a)–(b) from `outputs/exp_01_grid_validation/grid_heatmap.csv`, (c)–(d) from `outputs/exp_18/boundary.csv`.

7.3.3 A resolution condition, and what really happens at small t

At small t the likelihood $\ell_t(x_i \mid a_i)$ is a narrow Gaussian of width $\sqrt{\Delta_t}/e^{-t}$. If that width is comparable to the grid spacing, the integrand is under-resolved and the quadrature is meaningless however fine the rest looks. The working condition used here is $\sqrt{2t} \gtrsim 3\Delta u$.

Everything gets harder as $t \to 0$, and it is worth being precise about why, because the usual one-line explanation is wrong. It is often said that the score diverges as $\Delta_t \to 0$ because (2.2) carries a factor $1/\Delta_t$. It generally does not diverge: the numerator $x - e^{-t}\langle a \rangle_{x,t}$ vanishes at the same order, and for a P_0 with a continuously differentiable density $S(x,t) \to \nabla \log P_0(x)$, a finite limit. Four separate effects are at work, and only the first is a genuine mathematical singularity.

1. **A real singularity, for one family only.** The uniform-innovation chain has a compactly supported P_0, so $\nabla \log P_0$ is unbounded at the support boundary. The Laplace chain has a density with a kink, so its score has a jump discontinuity but stays bounded. The Gaussian and mixture chains have scores bounded on compacta.

2. **Numerical cancellation.** The implementation forms a difference of two nearly equal quantities and divides by $\Delta_t \approx 2t$, so relative error is amplified by $O(1/t)$ no matter how accurate the posterior mean is.

3. **Under-resolution.** The likelihood factor has width $\sqrt{\Delta_t}$; once that is comparable to h the quadrature integrates it with $O(1)$ points. That is the condition stated just above.

4. **Integrator stiffness.** The reverse drift's Jacobian scales like $1/\Delta_t$, so the step size must shrink as $t \to 0$. This is why `src/reverse.py` → `time_grid` is geometric rather than uniform.

The practical conclusion — stop at $t_{\min} > 0$, and treat the small-t end of every table with the most suspicion — is unchanged. The reason for it is not a divergence of the score.

7.4 Stability

Messages are products of many small numbers and underflow quickly. Two standard devices: work in the log domain where possible with per-row maximum subtraction, and renormalise each message to unit quadrature mass after every step, accumulating the discarded constants so the evidence is recovered exactly.

Implementation

Both appear in `src/bp_grid.py` → `grid_bp`: `log_ell -= log_ell.max(...)` before exponentiating, and an explicit mass check that raises `FloatingPointError` if a message loses all its mass rather than silently returning garbage.

7.5 The stopping-time problem

Because integration stops at $t_{\min} > 0$, what a sampler produces is a draw from $P_{t_{\min}}$, not from P_0. This sounds like a technicality and is not.

Independent Gaussian noise of variance v added to a quantity of variance q multiplies its excess kurtosis by $(q/(q+v))^2$. At $\rho = 0.85$ and $t_{\min} = 0.02$ this predicts a generated excess kurtosis of 1.910 where the clean chain has 3.0.

Assumption at work

This project measured exactly 1.91 for the reference sampler and initially read it as a failure of the integrator to converge. It was not: the sampler was reproducing $P_{t_{\min}}$ correctly and being compared against the wrong target.

The obvious repair — apply a final denoising step and report $\langle a \rangle_{x,t}$ instead of x — *inverts* the bias rather than removing it. The posterior mean of a heavy-tailed prior is a shrinkage operator: it pulls small values towards zero harder than large ones, so the law of the posterior mean is *more* peaked than P_0. Measured overshoot: 3.73, 4.34, 4.74 at $t_{\min} = 0.02, 0.05, 0.10$ against a true 3.0 — and identical at $N_g = 401$ and $N_g = 801$, which rules out grid resolution as the cause.

The correct treatment is to compare like with like: draw from P_0, noise it forward to the same $t_{\min}$, and compare. Both sides are then samples of the same law, no estimator sits between the sampler and the metric, and the target moments follow in closed form.

Implementation

`experiments/exp_16_sampling_validation.py` → `reference_at` builds the forward-noised reference; `target_innovation_moments` gives the closed-form target. The docstrings there record both failed designs, since the reasoning is easier to follow than to rediscover.

7.6 Running on a GPU

The recursion is a sequence of dense matrix products, which suits a GPU. Rather than write a second implementation, `src/backend.py` parameterises the existing one by its array module, so CPU and GPU execute the same code.

> Two implementations of an exact algorithm is a bad trade. They agree on the easy cases and drift where nobody is looking, and the exactness claims of §5.8 would then hold for one device and be assumed for the other.

Implementation

`tests/test_backend_parity.py` gates every GPU number: device agreement, the closed-form Gaussian check re-run on device, float64 preservation, and invariance to batch width. Measured on an NVIDIA H200: posterior means agree to 1.8×10^{-16}–8.0×10^{-16}, variances to 9.3×10^{-15}, and the GPU reproduces the closed-form Gaussian score to 4.2×10^{-16}. `hpc/bocconi_gpu.sbatch` → runs that suite before any sweep and exits if it fails.

Assumption at work

The availability probe originally tested that a GPU array could be *allocated*. Allocation succeeds without cuBLAS loaded, so on a misconfigured node the check passed and every matrix product then failed inside the recursion. It now probes an actual matrix product. A capability check must exercise the operation that will be used, not a cheaper proxy.

Chapter 8

Estimating the transition kernel

Everything so far assumed the chain is known. This chapter removes that assumption: we observe only noised sequences and must estimate K from them, never seeing a clean sample.

8.1 The objective

Write θ for the kernel parameters. The observed-data log-likelihood is

$$\mathcal{L}(\theta) \;=\; \sum_{\mu=1}^{M} \log P_{t_\mu,\theta}(x^\mu), \qquad P_{t,\theta}(x) = \int \mathrm{d}a\; P_0^\theta(a)\, P(x \mid a, t). \tag{8.1}$$

The integral is the same marginalisation Chapter 5 made tractable — it is the evidence, and BP already returns it.

> This is a latent-variable problem in the textbook sense: the clean sequence a is the latent variable, the noised x is observed, and the parameters live in the prior. What is unusual is that the latent structure is a tree, so the quantities EM needs are available *exactly* rather than by approximation — which is not the typical situation.

8.2 Expectation maximisation

EM alternates two steps. Given a current $\theta^{(r)}$, form

$$\mathcal{Q}(\theta \mid \theta^{(r)}) = \sum_{\mu} \mathbb{E}_{\theta^{(r)}}\!\left[\log P_\theta(a, x^\mu) \,\middle|\, x^\mu\right], \tag{8.2}$$

then set $\theta^{(r+1)} = \arg\max_\theta \mathcal{Q}$.

Proposition 8.1 (Monotone ascent)**.** $\mathcal{L}(\theta^{(r+1)}) \geq \mathcal{L}(\theta^{(r)})$.

Proof. For any θ, write $\log P_\theta(x) = \mathcal{Q}(\theta \mid \theta^{(r)}) - H(\theta \mid \theta^{(r)})$ with $H(\theta \mid \theta^{(r)}) = \mathbb{E}_{\theta^{(r)}}[\log P_\theta(a \mid x) \mid x]$. By Gibbs' inequality $H(\theta \mid \theta^{(r)}) \leq H(\theta^{(r)} \mid \theta^{(r)})$ for every θ, since the difference is a Kullback–Leibler divergence. Choosing $\theta^{(r+1)}$ to increase $\mathcal{Q}$ therefore increases $\mathcal{L}$ by at least as much. □

> Monotonicity is the reason EM is trustworthy without a step size. It is also the sharpest available test of an implementation: an error anywhere — in the recursion, the accumulation, or the maximisation — generically breaks it. In every run reported in this project the violation is exactly zero.

8.3 The expectation step is exact

In a general latent-variable model (8.2) is intractable and one resorts to a variational or sampled approximation, losing monotonicity with it. Here the posterior is a tree, so Proposition 5.1 gives the required expectations exactly.

The complete-data log-likelihood is a sum over edges, so the *θ-dependent part* of $\mathcal{Q}$ — it also contains a site-one term and a likelihood term, both θ-free and therefore additive constants — depends on the data only through

$$\boxed{\Xi_{kl} \;=\; \sum_{\mu=1}^{M}\sum_{i=2}^{L} P(a_{i-1}=u_k,\; a_i=u_l \,|\, x^\mu)\,,} \tag{8.3}$$

the expected transition mass between grid points, accumulated from the pairwise marginals (5.5).

> Ξ is the continuum analogue of the expected transition counts of Baum–Welch. In the discrete case it literally counts how often the chain was inferred to move from one state to another; here it is a mass on pairs of grid points.

Assumption at work

Sufficiency holds only under four conditions: a *homogeneous* kernel shared across sites and sequences, a fixed grid, an initial law excluded from θ, and θ-free likelihood factors. The last is what licenses observations at several noise levels to *add* into one Ξ. Relax any of them and (8.3) is no longer sufficient.

Note also what sufficiency does and does not buy. The maximisation step never revisits the data, so it costs $O(PN_{\mathrm{g}}^2)$ independently of M. But *forming* Ξ costs $O(MLN_{\mathrm{g}}^2)$ per iteration and $O(N_{\mathrm{g}}^2)$ storage. "The whole E-step is one matrix" is a statement about sufficiency, not about inference being cheap.

Implementation

`src/em.py` → `e_step` accumulates (8.3); `src/em.py` → `ExpectedStatistics` holds it; `src/em.py` → `fit_em` is the driver with the monotonicity trace. `src/em.py` → `clean_statistics` is the $t \to 0$ limit, which needs no BP at all — the clean-data case Marc's original suggestion referred to.

8.4 Fisher's identity: a gradient without differentiating the recursion

There is a second route to the same information. Fisher's identity states

$$\boxed{\nabla_\theta \mathcal{L}(\theta) = \sum_{\mu} \mathbb{E}_\theta[\nabla_\theta \log P_\theta(a, x^\mu) \,|\, x^\mu] = \big\langle \Xi(\theta),\; \nabla_\theta \log K_\theta \big\rangle.} \tag{8.4}$$

Proof. $\nabla \log P(x) = \nabla P(x)/P(x)$ and $\nabla P(x) = \int \mathrm{d}a\, \nabla P(a,x) = \int \mathrm{d}a\, P(a,x) \nabla \log P(a,x)$; dividing by $P(x)$ turns $P(a,x)/P(x)$ into the posterior. The second equality follows because $\log P_\theta(a,x)$ depends on θ only through the edge terms. □

> This says something practically important. BP is not a computation graph to be backpropagated through — it is the oracle that supplies the conditional expectation. The exact gradient of the *marginal* likelihood comes out of one forward–backward pass. Advice received elsewhere on this project held that EM here would require moving the recursion into an automatic-differentiation framework; that is not so, and the entire implementation is plain array arithmetic.

Assumption at work

Fisher's identity is *not* EM. It licenses gradient ascent on $\mathcal{L}$; EM maximises $\mathcal{Q}$ at each step. The two are different algorithms and neither dominates. Measured here: on the smooth Gaussian kernel gradient ascent reaches EM's optimum to 2×10^{-9} nats while needing a tuned step size (at too large a step it diverges outright); on the Laplace kernel it attains a *higher* likelihood than EM, because there the exact maximiser of $\mathcal{Q}$ is a lattice artefact of the discretisation.

Implementation

`src/em.py` → `q_gradient` implements (8.4); `experiments/exp_08_gradient_vs_exact_mstep.py` → is the comparison. The identity is checked against a central finite difference of the exact evidence in `tests/test_em_bp.py`, agreeing to a relative 1.5×10^{-9}.

8.5 The kernel families

class	free parameters	maximisation step
`GaussianAR1Kernel`	(ρ, q), 2	closed form: belief-weighted Yule–Walker
`LaplaceAR1Kernel`	(ρ, b), 2	closed form, via a weighted median
`MixtureInnovationKernel`	ρ and C triples, $3C$	inner ECM sweeps
`MDNKernel`	network weights	gradient, with backtracking

The mixture kernel is the one used for the headline results,

$$K_\theta(a' \mid a) = \sum_{c=1}^{C} \pi_c \mathcal{N}\big(a' - \rho a;\ \nu_c,\ s_c^2\big)\,, \tag{8.5}$$

because it keeps the linear autoregression while leaving the innovation law essentially free. It has never been shown the density it is asked to recover.

Counting the parameters. At $C = 4$: ρ, four weights, four locations, four scales — thirteen numbers, subject to $\sum_c \pi_c = 1$, hence **twelve free**.

Assumption at work

Two subtleties in that count. First, the zero-mean condition $\sum_c \pi_c \nu_c = 0$ is *not* enforced: imposing it by reprojection is not a maximisation step and broke monotone ascent in testing, so it is monitored as a diagnostic instead (it lands at the 10^{-3} level). A reader who assumed the model class of §3.2.2 literally would compute eleven. Second, and consequently, Proposition 6.1 applies to the *true* kernel but not automatically to the estimated one, whose innovation is not exactly centred.

Third: the mixture M-step is not the exact maximiser of $\mathcal{Q}$. The complete-data problem carries a second latent variable — the mixture label — so the step is itself iterative, run as four inner conditional-maximisation sweeps. That makes the algorithm *generalised* EM: monotone ascent is preserved, exact maximisation is not.

Implementation

`src/kernels.py` → `MixtureInnovationKernel` — see its `m_step`, whose `n_inner=4` is the ECM sweep count. `theta` exposes the flat parameter vector, which is what the twelve-versus-thirteen count is read from.

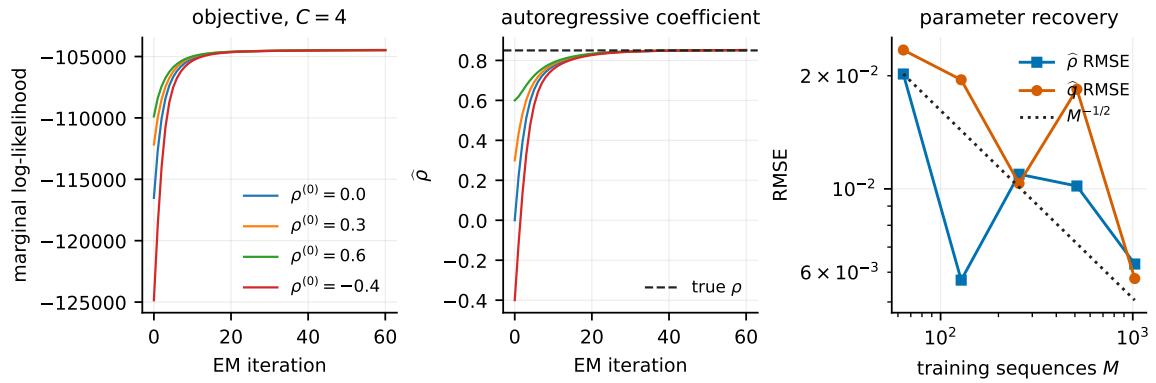


Figure 8.1: Left: the marginal log-likelihood by iteration at $C = 4$, $M = 512$, from four initialisations of ρ; the largest decrease across iterations is zero in all four runs, and in the four $C = 8$ runs not shown. Centre: the estimated autoregressive coefficient over the same runs, reaching $[0.8517, 0.8520]$ against a truth of 0.85 from initialisations spanning $[-0.4, 0.6]$ — which is the direct evidence that ρ is estimated rather than supplied. (The eight $C \in \{4, 8\}$ runs together span $[0.8517, 0.8529]$.) Right: RMSE of $\widehat{\rho}$ and of the innovation variance against the number of training sequences, with an $M^{-1/2}$ reference. Sources `outputs/exp_18/em_trace.csv`, `outputs/exp_06_em_parameter_recovery/em_rate.csv`.

8.6 Identifiability

Two claims must be separated, and confusing them overstates what is proved.

Distribution level. Gaussian convolution at finite t is injective. In characteristic-function terms $\varphi_{t,\theta}(\xi) = \varphi_\theta(e^{-t}\xi)\, e^{-\Delta_t \|\xi\|^2/2}$; the Gaussian factor has no zeros so it divides out, and $e^{-t} > 0$ makes $\xi \mapsto e^{-t}\xi$ a bijection. So P_t determines P_0.

Parameter level. That P_0 is determined does *not* by itself determine K. That additionally requires: the initial law fixed or separately identifiable; the transition family identifiable as a parametric family; the mixture treated modulo label permutation; the mixture parameters in the interior — all $\pi_c > 0$ with the (ν_c, s_c) distinct, which an over-specified C need not respect; and μ of full support, since K is pinned down only there. We assume these rather than prove them.

Injective is not the same as stable. This is Gaussian deconvolution, whose inverse is unbounded, so identifiability survives at any finite t while the *information* degrades rapidly — and unevenly. Measuring that unevenness needs care about what is being compared. In the Gaussian model with parameters (ρ, q) the innovation *scale* q falls by a factor 142 between $t = 0.05$ and $t = 1.6$ while the correlation falls by 26, but both of those are second-order quantities: q is a scale, not a shape, so the ratio says nothing about higher-order structure.

A genuine shape coordinate is needed. On a generalised-Gaussian innovation $\varphi_\beta(e) \propto \exp(-(|e|/s)^\beta)$ with s chosen so the variance is exactly q, β moves the fourth moment at fixed second moment. Reporting the *efficient* information for each coordinate — projecting out the other two, since the diagonal ignores their coupling — the decay across the same range is 56–87 for the correlation, 235–522 for the variance and 8.7×10^3–6.8×10^4 for the shape. Higher-order structure is therefore not five times more fragile but 112 to 1222 times, which is exactly the structure this project cares about.

Implementation

`experiments/exp_06_em_parameter_recovery.py` → measures the information decay (`price_of_noising.csv`) and the recovery of an unknown innovation law from noisy data alone.

Sources

EM is Dempster et al. [21]; the convergence analysis, including what monotone ascent does and does not guarantee, is Wu [22]. The generalised and conditional-maximisation variants used here — which is what the mixture M-step actually is — are Meng and Rubin [23]. Fisher's identity for the gradient of the marginal likelihood goes back to Fisher [24] and is given in the latent-variable form used here by Louis [25] and Cappé et al. [17]. The transition-count statistic $\boldsymbol{\Xi}$ is the continuum analogue of the one in Baum et al. [13]. Identifiability of finite mixtures, which Assumption 1 of the note leans on, is Teicher [26] and Yakowitz and Spragins [27].

Chapter 9

The frozen batch: what is settled and what is not

This chapter is the experimental record. It replaces an earlier version that reported results from runs at three different EM iteration budgets, three different replicate counts and two values of ρ — numbers that could not go in one table, and several of which turned out to be artefacts of those settings rather than of the model. Chapter 11 records what those were and how they were caught.

Everything below comes from a single configuration, defined once in `experiments/frozen_config.py` and imported by every experiment.

9.1 The configuration

$\rho = 0.85$	one value everywhere; the earlier 0.8 runs are retired
$L = 32$	chain length, which is also the ambient dimension
$N_{\mathrm{g}} = 401$, $A = 8$	read off the grid/domain sweep, §9.2
twelve noise levels	log-spaced on $[0.05, 3.0]$
sixteen replicates	everywhere
$M \in \{32, \ldots, 4096\}$	seven doublings
256 held-out sequences	drawn from a seed disjoint from every training seed
EM stopping	relative 10^{-9} on the log-evidence, cap 400; **never** on ρ
ring	$T = 12$, so $L = 24$

Assumption at work

A frozen configuration is only as good as its reach, and this one has now failed that test twice. Four experiments carried private replicate knobs that never read `n_seeds`, and every one of them reported `is_frozen: true` while running at three, four or eight replicates (§11.2). The same disease then turned up in the iteration budget: `em_max_iters` and `em_shape_tol` are declared in the config and read by *nothing*, while `exp_07` passed a literal 120 (§11.1). Both were invisible to the provenance stamp, which compares the config against itself. `tools/check_replicates.py` now parses the settings dictionaries for both classes of knob.

The general lesson is worth stating once: a frozen configuration is a claim about what the experiments read, and only a reader-side check can test that claim. Declaring a value is not the same as consuming it.

9.2 Settled: the numerical foundation

The discretisation is controlled on two axes, separately. Truncation — mass outside $[-A, A]$ discarded rather than transported — responds to the domain and not the resolution: the residual falls from 3×10^{-4} at $A = 4$ to 1.4×10^{-8} at $A = 8$, essentially independently of N_{g}. Quadrature error falls by a factor of four per halving of h, the predicted second order for the trapezoidal rule. Neither sweep alone certifies the other, which is why both are reported.

The pipeline is verified where the answer is known. At $N_{\mathrm{g}} = 401$, $A = 8$ the discretised recursion reproduces the Gaussian closed form to 9.2×10^{-15} relative, and agrees with brute-force enumeration of the discretised posterior — summing over all N_{g}^{L} configurations on a short chain, sharing no code with the message passing — to 1.6×10^{-14} on posterior means and 1.8×10^{-15} on the log-evidence. Two independent routes, not one self-consistent one.

What moment closure costs. By Proposition 6.1 the gap between moment-projected and full sum-product is exactly the price of replacing the prior by its covariance-matched Gaussian. Measured on the Laplace chain, the median relative score error falls from 0.241 at $t = 0.05$ to 7.3×10^{-6} at $t = 3$ — a factor of 3.3×10^{4} across the schedule, as noising Gaussianises the posterior and the innovation's shape stops mattering.

9.3 Settled: learning the kernel

Both optimisation routes reach the same place. On the Gaussian kernel, gradient ascent via Fisher's identity reaches the exact M-step's optimum to 0.021 nats at its best step size, with slightly better parameter errors (0.0040 against 0.0053 on ρ). The difference is robustness, not accuracy: EM's monotonicity violation is 0.0 exactly and it needs no step size, while gradient ascent at $\eta = 2.0$ diverges (ρ error 0.41, monotonicity violation 1641) and at $\eta = 8.0$ is unstable. That is the whole content of the comparison, and it is why every rung above uses EM.

The gradient is exact. Fisher's identity is verified against finite differences of the discretised evidence to $\sim 10^{-9}$. This check matters more than it looks: a fixed-point residual near zero is evidence of a stationary point only if the derivative defining it is right, and a sign error in exactly that derivative once survived every internal diagnostic (§11.5).

9.4 Settled: what the structure buys

Relative denoising error against grid BP under the true kernel, averaged over the noise schedule, with the network's parameterisation chosen on a disjoint validation split:

M	network	EM–BP (12 free)	ratio
32	0.6613 ± 0.0026	$\mathbf{0.0703 \pm 0.0051}$	9.4
64	0.5896 ± 0.0029	$\mathbf{0.0541 \pm 0.0035}$	10.9
128	0.4942 ± 0.0015	$\mathbf{0.0361 \pm 0.0024}$	13.7
256	0.3473 ± 0.0012	$\mathbf{0.0259 \pm 0.0014}$	13.4
512	0.2359 ± 0.0010	$\mathbf{0.0233 \pm 0.0010}$	10.1
1024	0.1685 ± 0.0006	$\mathbf{0.0185 \pm 0.0011}$	9.1
2048	0.1364 ± 0.0007	$\mathbf{0.0172 \pm 0.0008}$	7.9

All sixteen seeds, all seven doublings, $\pm$ one standard error over seeds. The validation-selected parameterisation agrees with the test-set optimum in 99.4% of the 1,344 cells, so the oracle it replaces is worth nothing measurable. The network at $M = 2048$ (0.136) has not

reached the error the estimator attains at $M = 32$ (0.070); no crossing is observed anywhere in the range, so no data-equivalence factor is quoted.

The ratio is not monotone — it rises to 13.7 at $M = 128$ and falls to 7.9 at 2048 — and that shape should not be over-read. Both arms improve with M; the estimator starts so far ahead that the ratio is a quotient of two moving quantities, and nothing here identifies a mechanism for the turn. What the table supports is the statement that the margin is an order of magnitude across five doublings, not a claim about how it scales.

Assumption at work

These sixteen seeds ran at 120 EM iterations, because `exp_07` hard-coded that budget while `FROZEN.em_max_iters` said 400 and nothing read it (§11.1). The rerun at the configured budget is Bocconi array `628943`.

The obvious prediction about that rerun was wrong, and the way it was wrong is the interesting part. The reasoning was: the shape settles at a median of 229 updates, so a run stopped at 120 is short of convergence, so the estimator is handicapped and the ratios are a lower bound. On the first six seeds the direction *reverses with sample size*. Held-out score error of the estimator, same six seeds, 120 against 400 iterations:

M	EM at 120	EM at 400	
32	0.0585	0.0673	worse by 15%
64	0.0501	0.0626	worse by 25%
256	0.0283	0.0295	worse by 4%
512	0.0223	0.0200	better by 10%
2048	0.0158	0.0125	better by 21%

Running the optimiser *longer* makes the estimator *worse* at small M and better at large M, so no fixed budget is neutral between the two arms of the comparison.

The mechanism, measured rather than inferred. An instrumented run (`experiments/exp_29_em_overfitting.py`) records the training marginal log-evidence and the held-out score error on the *same* iterates, which is what separates overfitting from a bad optimum: overfitting means the training objective keeps improving while generalisation turns around, whereas a bad optimum would show both stalling.

M	training $\mathcal{L}$ monotone ↑	held-out best at	cost of running to 400
32	yes	iteration 130	+2.8%
64	yes	iteration 40	+46.0%
2048	yes	iteration 250	+8.6%

The training log-evidence increases monotonically to the last iterate at every size — as EM guarantees it must — while the held-out error has an *interior minimum* at every size. That is overfitting, and the diagnosis is now a measurement rather than a plausible story. Note what the third row says: the interior optimum is at 250 even at $M = 2048$. The effect is **not** confined to small samples, and the earlier reading — "120 wins below the crossover, 400 wins above" — was an artefact of comparing two arbitrary points either side of an optimum that moves. A cap of 400 overshoots at every size tested.

Two consequences. First, "run it to convergence" is not automatically the honest choice. The iteration count is a hyperparameter with a bias–variance trade-off like any other, and fixing it a priori quietly favours one arm; picking it on the test set would be exactly the

sin §11.3 records. It is now chosen on the validation bundle, per noise level, symmetrically with the network's parameterisation, which is what `629091` runs.
Second — and this is why it is in this chapter rather than a footnote — this is the memorisation–generalisation question of §2.3 appearing in a model with twelve free parameters and no network anywhere. The training objective and the generalisation objective come apart, early stopping is doing implicit regularisation, and all of it is visible in a setting where the target score is computable in closed form. That is precisely what this setting was built for, and it is the most promising thing found in this batch.

Still not claimed. That the optimal stopping point scales with M in any particular way. The three sizes here give 130, 40 and 250, which is not monotone, and three points chosen for a diagnostic are not a curve. Reading a scaling law off them would repeat §11.3 exactly.

9.5 Settled: structure the marginals cannot see

Chapter 10 in full. The result in one line: across 1,344 cells the per-frame arm's estimate of $p(\psi)$ moves 0.0000 in total variation from its uniform initialisation, and across the 480 cells of the single-rotation variant its profile likelihood in ψ has range 0.00×10^0 nats — a machine-precision zero, which is what Theorem 10.1 requires. The joint arm recovers the rotation to $0.15°$ at $t = 0.05$, degrading to $7.09°$ at $t = 1.43$ and returning to the null by $t = 3$.

9.6 Settled: where the Markov assumption fails

Two contamination mechanisms, both with an exact reference, from the complete ten-task run `627165`. `ratio_to_em` is the baseline's relative score error over EM–BP's, so > 1 means the estimator wins.

	global latent, rank-one, strength β				
	0.00	0.10	0.25	0.50	1.00
Gaussian, vs CNN	18.3	17.2	6.99	3.36	2.22
Laplace, vs CNN	9.90	10.6	6.50	3.05	2.12
	long-range precision coupling, strength γ				
	0.00	0.05	0.10	0.20	0.40
Gaussian, vs CNN	24.1	1.19	**0.98**	0.86	0.77
Gaussian, vs MLP	37.0	1.79	1.23	0.96	0.83

The estimator survives rank-one contamination essentially intact — still winning at $\beta = 1$, where half the marginal variance is a shared constant — and **inverts at** $\gamma \approx 0.10$ under long-range coupling. The mechanism is exactly what the structure predicts: a global latent makes the score residual rank one, which a chain absorbs; long-range coupling is not low-rank and cannot be represented. This is the honest scope statement, and it is backed by a complete run.

Assumption at work

These runs predate `frozen_config.py` and are *not* at the frozen settings. They are reported here, in the development document, and are deliberately not quoted in the note or the workshop paper. The qualitative conclusion — rank-one survivable, long-range not — is robust to the settings; the specific ratios are not to be pooled with §9.4.

9.7 In progress

Four of the five reruns listed in the previous version have landed and moved into the sections above; what follows is what they said and what is still out.

Landed: the parametric rate (`628576`). At four replicates the fitted log–log slopes were -0.196 for ρ and -0.700 for q against a predicted -0.500, with a non-monotone ρ curve — not a confirmation of $M^{-1/2}$, and the claim was pulled from the note rather than caveated. At sixteen the variance slope is -0.519 ($r = -0.99$, monotone), which is the rate; ρ is monotone but shallower at -0.273. Half the claim is restored, half is not, and the note says so rather than averaging them into a verdict.

Landed: clean against channel (`628600`). Two slownesses, and they are independent. *Optimisation*: coordinates of one kernel settle at very different rates — median 80 updates for ρ, 68 for q, 229 for the excess kurtosis (`628601`, below). *Statistics*: fitting through the channel costs rate, not merely a constant.

log–log slope, sixteen replicates	ρ	q
clean transition pairs	-0.52	-0.50
through the channel	-0.44	-0.36

The clean arm sits on the parametric rate on both coordinates, which is the check that the estimator is not itself the bottleneck. It is also flatly incompatible with the withdrawn discretisation-floor reading of §11.3: the clean arm falls by a factor of 3.4 on ρ and 4.3 on q across the range, so there is no floor to see.

Landed: shape convergence at sixteen seeds (`628601`). The settling table above. Zero monotonicity violations across all sixteen traces. The kurtosis figure has a long tail — up to 638 updates in the slowest seed — which is what makes a 400 cap a decision rather than a formality.

Landed: sample efficiency at sixteen seeds (`628571`). §9.4, with the iteration-budget caveat recorded there.

Landed, partly: the confining potential (Rung 4a, `629193`). Three attempts, and the third is still running, so this is worth setting out in order.

`628434_5` was killed by the six-hour wall having written nothing — 960 gradient fits at $\sim$0.2 s per step is roughly ten hours, and the experiment writes once at the end. Sharding by seed fixed that. `628942` then completed and produced nothing usable, for a reason that had nothing to do with the queue: `part_potential` generated two species at $\pm\omega$ while `fit_potential` scores every trajectory at the single scalar ψ it is handed, so half the data was evaluated under the wrong rotation. The estimator's escape was to collapse the ring. At $r_* \to 0$ the ring is rotation-invariant and so pays nothing for a wrong ψ; r_* pinned to the lower clip in 51.2% of cells, and the sweep reported the *marginal* arm beating the joint arm, which is the reverse of what this rung exists to show.

`629193` (one species, 600 steps, sixteen seeds, 960 cells) repairs that:

	λ rel. err. (med.)	r_* abs. err. (med.)	pinned at clip
joint, two species (`628942`)	1.844	0.950	51.2%
joint, one species (`629193`)	0.229	0.020	4.8%
marginal, one species	0.429	0.039	4.4%

The joint arm's error falls monotonically with sample size — 0.618, 0.336, 0.222, 0.181, 0.118 at $n = 128$ to 4096 — and that trend is reported here because it survives the caveat below.

Assumption at work

The arm *comparison* from `629193` is not reported, because the two arms were not held to the same convergence standard. At $t \leq 1$ the marginal arm hit the 600-step cap in 95% of cells against the joint arm's 57%, so "joint beats marginal" partly means "joint converged and marginal did not". The obvious control does not rescue it: only 13 of 400 pairs have both arms converged.

That this actually bites was measured rather than assumed. Refitting with the plateau test disabled (n=512, t=0.2216), the joint arm is identical to eight significant figures from step 600 onward, while the marginal arm keeps improving: λ error 0.482 at 600, 0.372 at 1200, 0.335 at 2400. So `629193` stopped the marginal arm roughly 30% short of its own optimum while the joint arm was fully converged — the same unmatched-budget error as comparing EM at a fixed iteration count against a network trained to a fixed length.

The cause was the stopping rule. `plateau_tol` was an *absolute* tolerance on an objective that sums over sequences, hence eight times stricter at n=4096 than at n=512: a convergence criterion varying along the very axis this rung reports its scaling on. It is now per-sequence, pinned by `tests/test_ring.py::test_stopping_rule_does_not_depend_on_sample_size`, which replicates one dataset fourfold and asserts the fit stops at the same step.

Still out.

- **Rung 4a at a matched convergence budget** (`629962`), eight seeds split by sample size into sixteen shards, at a 3000-step cap under the per-sequence tolerance. Until it lands, the joint-versus-marginal margin above stays out of every document.
- **Sample efficiency with both budgets selected** (`629985`), which will replace the table in §9.4. `629091` selected EM-BP's iteration count on validation while the network trained to a fixed 20,000 steps — the same fairness error that validation selection was introduced to remove, relocated rather than fixed. Training length is a bias/variance knob for both arms: EM's held-out error has an interior minimum at every sample size (§9.4), and SGD's does too. Selecting one budget while pinning the other favours the selected arm, and here it favoured us. The network's training length is now chosen on the same bundle, jointly with its parameterisation. Each arm has one selected quantity; the network selects over a two-dimensional grid and EM-BP over one.

9.8 Withdrawn, and not replaced

Recorded in full in Chapter 11; listed here so that nothing in this chapter is read as still standing.

- **The capacity attribution.** That enlarging the innovation mixture buys generative fidelity. Roughly 96% of the apparent effect was convergence rate at a fixed iteration budget. The rerun that would settle it lost all but one of its array tasks, so one cell survives and the claim stands withdrawn and unreplaced (§11.1).
- **The discretisation floor.** That the clean-arm variance error was floored by the grid. It was under-replicated, not floored, and the grid contributes a constant 4.4×10^{-4} — an order of magnitude too small to produce the effect (§11.3).

- **Everything downstream of the superseded generation protocol**, including the four-family comparison at matched covariance (§11.4).

9.9 Not started

The architectural question. Sum-product on a chain suggests a network that is narrow but as deep as twice the chain, unlike the tree-structured case where the natural depth is the tree's [18, 28]. Which architectures exploit Markovianity, and at what depth, is what this setting was built to measure, and is the obvious continuation. The receptive-field sweep that would begin it exists (`exp_12`) but selected its radius on the evaluation set, so its numbers are exploratory and are not reported anywhere.

Chapter 10

The rotating ring: structure the marginals cannot see

Every model so far has been the AR(1) chain, where what is being estimated — the autoregressive coefficient, the innovation density — shows up in the data's low-order statistics. Look at a histogram of a_i and of $a_i a_{i-1}$ and you have most of it. This chapter develops a model where that is false, and where it is false for a reason one can prove rather than observe.

The model is Problem 1 of the three panels on Mézard's board, developed in `research/board-3problems`. What follows is self-contained; the port into this project's package is `src/ring.py`, and §10.7 records how it was checked against the independently audited original.

10.1 The model

Two clocks, and keeping them apart is the whole setup. *Internal* time $u = 0, \dots, T-1$ runs along a trajectory. *Diffusion* time t is the noise level. One sample is a **whole trajectory**, so the ambient dimension is $L = 2T$ and the channel hits every frame at once.

Three independent latent variables:

$$r_0 \sim p_\lambda(\rho) \propto \rho\, e^{-(\rho - r_*)^2/2\lambda}, \qquad \theta_0 \sim \mathrm{Unif}[0, 2\pi), \qquad \psi \sim p(\psi), \tag{10.1}$$

and then a rotating random walk in the plane,

$$z_0 = r_0(\cos\theta_0, \sin\theta_0), \qquad z_{u+1} = R_\psi z_u + \sigma\,\eta_u, \qquad \eta_u \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2) \text{ i.i.d.} \tag{10.2}$$

> The factor ρ in p_λ is the two-dimensional area element, not part of the physics: the density of the *point* $z_0 \in \mathbb{R}^2$ is $\propto e^{-(\|z\| - r_*)^2/2\lambda}$, and converting to a radial marginal picks up the Jacobian. This is why the mode of p_λ sits slightly outside r_*.

(10.2) is the discrete form of the pair written on the board, $\mathrm{d}r = (r_* - r)\,\mathrm{d}u + \Gamma\,\mathrm{d}B_u$ together with $\dot\theta = \omega$ — that is, a **quadratic confining potential** $V(r) = (r - r_*)^2/2\lambda$ holding the radius near r_*, plus a steady rotation. So the model has two kinds of parameter, and the point of this chapter is that they behave completely differently: λ and r_* describe the well, and ψ describes the dynamics.

10.2 The rotation is a gauge

Let $U_\psi = \mathrm{blockdiag}(R_{-u\psi})_{u=0}^{T-1}$, which rotates frame u backwards by $u\psi$. It is orthogonal, so it commutes with the isotropic channel: if $\mathbf{x} = e^{-t}\mathbf{a} + \sqrt{\Delta_t}\,\xi$ then $U_\psi \mathbf{x} = e^{-t} U_\psi \mathbf{a} + \sqrt{\Delta_t}\, U_\psi \xi$ and $U_\psi \xi \overset{d}{=} \xi$. Applying it to (10.2) turns the rotating walk into a driftless one, so

$$P_t(\mathbf{x} \mid \psi) = P_t^0(U_\psi \mathbf{x}), \tag{10.3}$$

with P_t^0 the $\psi = 0$ density. At known ψ the rotation costs nothing statistically — it is a change of coordinates.

This is why the model is tractable at any T despite being genuinely non-Gaussian. One does not need a separate calculation per rotation angle; one gauges the data and evaluates a single density.

10.3 The $\psi = 0$ density in closed form

Conditionally on z_0 the trajectory is Gaussian with a z_0-independent covariance, so P_0 is a two-parameter *location* mixture of one Gaussian. Write $X \in \mathbb{R}^{T\times 2}$ for the gauged trajectory, $K_{uv} = \min(u, v)$,

$$A_t = e^{-2t}\sigma^2 K + \Delta_t\, \mathbf{I}_T, \qquad g = A_t^{-1}\mathbf{1}, \qquad \kappa = \mathbf{1}^\top g, \qquad \beta = \|X^\top g\|.$$

Then

$$\log P_t^0(X) = -\tfrac{1}{2}\operatorname{tr}\Big(X^\top A_t^{-1} X\Big) \;+\; \log \mathcal{Z}_t(\beta) \;-\; \log|A_t| \;-\; \log C(\lambda, r_*), \tag{10.4}$$

with the radial integral and its normaliser

$$\mathcal{Z}_t(\beta) = \int_0^\infty \rho\, e^{-(\rho - r_*)^2/2\lambda}\, e^{-e^{-2t}\kappa\rho^2/2}\, I_0\big(e^{-t}\rho\beta\big)\,\mathrm{d}\rho, \qquad C(\lambda, r_*) = \int_0^\infty \rho\, e^{-(\rho-r_*)^2/2\lambda}\,\mathrm{d}\rho. \tag{10.5}$$

A quadratic form plus one one-dimensional Bessel integral. No network, no Monte Carlo, no curse of dimension — which is what makes this a reference rather than a benchmark.

Assumption at work

$C(\lambda, r_*)$ is **not optional**, and omitting it fails silently. The ring density in (10.1) is written unnormalised, and its normaliser depends on both parameters of the well. If one estimates ψ at fixed (λ, r_*), the term is a constant and cancels in every responsibility — which is exactly why it can sit missing for a long time without anything going wrong. The moment one estimates λ, it stops being a constant: a wider well simply contains more mass, so the likelihood increases without bound in λ and the estimate pins to the top of whatever grid it is given. Measured on this implementation, with the truth at $\lambda = 0.05$:

λ	0.02	0.03	0.05	0.08	0.11	0.15
without C	−1870	−1435	−911	−476	−218	**0**
with C	−43.0	−13.9	**−0.6**	−35.3	−96.0	−190

The upper row is smooth, confident and wrong. `tests/test_ring.py` pins both directions: that the profile likelihood peaks at the truth, *and* that it does not when the normaliser is removed — so deleting the term cannot make the suite greener.

Assumption at work

The count differs between the two likelihoods, and this is not a slip. The *joint* model invokes the ring density once, for z_0, and propagates; it subtracts $\log C$ once. The per-frame *marginal* model treats the frames as independent, so it invokes the ring density T times and must subtract $T \log C$. The first version of this code fixed the joint likelihood and left the marginal one wrong, and the error surfaced only when the blind arm of the potential experiment returned $\hat\lambda = 3.5 \times 10^6$.

10.4 Marginal blindness

Theorem 10.1 (Marginal blindness)**.** *For the model* (10.1)–(10.2) *with uniform initial phase, every single-frame marginal is independent of ψ, at every noise level. Consequently a model built from per-frame marginals carries exactly zero Fisher information about the rotation.*

Proof. Unrolling (10.2),

$$z_u = R_\psi^u z_0 + \sigma \sum_{k=1}^{u} R_\psi^{u-k} \eta_k.$$

Each η_k is isotropic Gaussian and a rotation of an isotropic Gaussian has the same law, so $R_\psi^{u-k}\eta_k \overset{d}{=} \eta_k$, independently across k. And θ_0 is uniform on the circle, so z_0 has a rotationally invariant law and $R_\psi^u z_0 \overset{d}{=} z_0$. The two terms are independent, hence

$$z_u \overset{d}{=} z_0 + \sigma\sqrt{u}\,\xi, \qquad \xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2)\,, \tag{10.6}$$

in which ψ does not appear. The channel acts coordinatewise and independently of ψ, so the noised frame is $x_u \overset{d}{=} e^{-t}(z_0 + \sigma\sqrt{u}\,\xi) + \sqrt{\Delta_t}\,\xi'$, again free of ψ. Therefore $\partial_\psi \log P_t(x_u) \equiv 0$ for every u and every t, so each frame's Fisher information about ψ is exactly zero. A sum of per-frame log-densities cannot contain information that no term contains. □

It is worth being precise about what this does *not* say. The marginals are not uninformative in general — they determine λ, r_* and σ perfectly well, and the per-frame law is a visible, non-Gaussian ring whose radius grows as $\sqrt{u}$ by (10.6). They are blind only to the *rotation*. Nor does the theorem say the joint is easy: the joint carries the information, and extracting it is what the estimator does.

The sharp form of the statement is that this is not a matter of degree. A per-frame model is not *less* informative about the dynamics, or informative only at low noise. It carries zero, exactly, at every noise level. Radii are not evidence.

10.5 Three estimation targets on one model

The chapter's design follows from the two preceding sections. The same model carries a parameter the marginals can see and a parameter they provably cannot, so running both through the same estimator isolates the effect of the structure rather than of the method.

	target	free parameters	visible to marginals?
4a	the confining potential (λ, r_*)	two	**yes**
4b	a single rotation ψ	one	**no** (Theorem 10.1)
4c	a law over rotations $p(\psi)$	a distribution	**no**

Each is run in two arms differing only in which likelihood scores a trajectory: the joint (10.4), or the product of per-frame marginals. That makes the rung a 3×2 design whose diagonal is the claim.

4a, the potential. Gradient ascent on $(\log\lambda, r_*)$ from a random initialisation, with a numerical gradient — honest for two parameters, and it avoids a hand-derived derivative that could be wrong in the same silent way the normaliser was. This is the *easy* case, and it is easy for a reason one can state: by (10.6) frame u's radial law is the well convolved with a Gaussian of variance $\sigma^2 u$, so every frame carries information about it.

Assumption at work

The iteration budget is doing real work here, exactly as it does for EM on the chain (Chapter 8). At 25 gradient steps this fit reports $\lambda = 0.075$ against a truth of 0.05 and looks perfectly well-behaved doing it; the likelihood plateaus near 150 steps and the estimate settles by 300 ($\lambda = 0.0493$, $r_* = 0.9966$ at $n = 512$). The experiment therefore runs to a plateau criterion with 300 as a cap, and records a `hit_cap` flag so that a fit which stopped because it ran out of budget cannot be read as one that converged.

4b, one rotation. Profile the likelihood over a grid of angles and refine parabolically through the peak, so the estimate is not limited by the grid. One scalar — fewer parameters than the potential — and by Theorem 10.1 no marginal carries any information about it.

4c, a law over rotations. EM for a mixture whose components are indexed by the rotation, on a truth of two species $p^*(\psi) = \frac{1}{2}\delta_{+\omega} + \frac{1}{2}\delta_{-\omega}$:

$$\text{E:}\quad r_\mu(\psi_k) \propto p(\psi_k)\, P_t(\mathbf{x}^\mu \mid \psi_k), \qquad \text{M:}\quad p(\psi_k) \leftarrow \frac{1}{M}\sum_\mu r_\mu(\psi_k).$$

The same sufficiency that makes $\boldsymbol{\Xi}$ a one-shot statistic on the chain appears here. The ψ-conditional log-likelihood does not depend on $p(\psi)$ — the thing being estimated — so the $M \times |\psi|$ matrix of log-densities is computed *once* and every EM iteration is a reweighting of it. The E-step touches the data one time and the M-step never revisits it.

The blind arm is not a strawman baseline; it is the theorem's own control. Its likelihood is constant in ψ, so its responsibilities equal $p(\psi)$ exactly, its M-step is the identity, and $p(\psi)$ can never leave its uniform initialisation. If the run shows anything else, the implementation is wrong rather than the theorem.

10.6 What the frozen batch found

Sixteen seeds, seven budgets, twelve noise levels, from `outputs/frozen/exp_28_ring_em`.

The blind arm returns exactly the null. Across the 1,344 cells of 4c the estimate moves 0.0000 in total variation from its uniform initialisation, and across the 480 cells of 4b the profile likelihood in ψ has range 0.00×10^0 nats. Not a small number — a machine-precision zero, everywhere, which is what a theorem predicting exactly no information requires. Reported as $\hat{\omega}$, the blind arm returns 90.00°, the value a uniform $p(\psi)$ gives, at every budget and every noise level.

The joint arm recovers the rotation, and loses it to the channel. Estimating a single angle, the error is $2.1 \pm 0.7°$ pooled over the whole schedule. Recovering the two-species law, the median error in ω against a truth of 30° and a null of 90°:

t	0.05	0.32	0.68	0.98	1.43	3.00
median $\lvert\hat{\omega} - \omega\rvert$	0.15°	0.24°	0.90°	2.86°	7.09°	58.8°

Information about the rotation survives until $t \approx 1$ and is gone by $t = 3$, where the estimate has returned to the null. That is the channel doing what a channel does, and it is visible here in a way it is not on the chain, because the null is known exactly.

The potential. [PENDING] — the 4a sweep runs 300 gradient steps per cell and had not completed when this was written. The preliminary result is that both arms recover (λ, r_*), which is the expected contrast with 4b: the well is visible to the marginals and the rotation is not.

10.7 The port, and how it was checked

`src/ring.py` is a port of code in `research/board-3problems`, which is audited there against independent polar quadrature (check `P1d`, the ψ-posterior against quadrature at 10^{-9}). A port is exactly the kind of change that can agree on the easy cases and drift somewhere nobody looks, so it is checked rather than assumed:

- Before the normalisation fix, the two implementations agreed **bit for bit**: maximum absolute difference 0.0 over a batch of 400 trajectories.
- After it, they differ by exactly $\log|A_t| + \log C$ — constant across the batch to 3.6×10^{-15}, and equal to the intended constant to 2.7×10^{-15}. So the change is provably the normalisation and nothing else.
- Marginal blindness is confirmed numerically as a distributional statement rather than a pathwise one: per-frame second moments agree across $\psi \in \{0, \pm 30°, 90°\}$ within four standard errors at $n = 40{,}000$, and match the closed form $\mathbb{E}\|z_0\|^2 + 2\sigma^2 u$.

Implementation

`src/ring.py` → `log_pt_conditional` is (10.4); `src/ring.py` → `log_pt_marginal` is the blind model, and takes no ψ argument because there is no ψ to take. `experiments/exp_28_ring_em.py` runs the three targets as four parts; `tests/test_ring.py` carries eleven checks including both directions of the normaliser.
One performance note, because it changed the experiment's feasibility rather than only its speed: `src/ring.py` → `log_pt_marginal` needs T different values of κ, one per frame, and originally rebuilt the whole Bessel table for each. Since I_0 depends only on $z = e^{-t}\beta\rho$ and not on κ, the expensive part can be computed once and shared across frames. That is an $11\times$ speed-up (522 ms → 48 ms per call) with identical output. The tell that the cost was in the quadrature and not the data was that the marginal likelihood took the same time at $n = 512$ and $n = 4096$.

Chapter 11

What we got wrong, and how we found out

This chapter exists so that the note and the workshop paper do not have to carry it. A reader of those documents should meet claims, not retractions of claims; a reader of this one should be able to see what was believed, why it was wrong, and — the part that matters for the next mistake — what diagnostic caught it.

Every entry follows the same shape: the claim as it was made, the correction, and the check that now exists so it cannot recur silently. Several of these were live in circulated drafts.

11.1 The iteration budget that manufactured a capacity effect

The claim. At $C = 4$ the estimator was an order of magnitude better pointwise than the convolutional baseline and yet generated a worse residual law; varying C improved both axes monotonically, which identified the deficit as a **capacity** limit of the innovation mixture.

What was wrong. Every run behind that account used `em_iters` $= 40$. EM on this kernel converges on ρ far faster than on the innovation *shape*: at $\rho = 0.85$, $M = 512$, $C = 8$ the fitted excess kurtosis reads 0.84 at 30 iterations, 1.85 at 60, 2.30 at 120 and 2.29 at 400, while ρ is flat from iteration $\sim$25. Monitoring ρ — which is the natural thing to plot — gives a converged-looking trace over a kernel that is nowhere near converged. Enlarging the mixture at a fixed iteration budget buys convergence *rate*, not representational capacity, and roughly 96% of the apparent capacity benefit was that.

What replaced it. Nothing yet, and this is worth stating plainly. The rerun that would settle it was submitted as Bocconi array `627164`, and **only array task 2 ever ran** — `sacct` lists `627164_2` and nothing else. One cell survives, $C = 2$ at one seed, in which the estimator is consistent with the reference (1.837 ± 0.061 against 1.894 ± 0.096, a gap of 1.20σ). That is suggestive that the capacity reading was indeed a convergence artefact — a two-component mixture would not have sufficed under the old account — but one cell at one capacity cannot replace a withdrawn claim. **The capacity attribution remains withdrawn and unreplaced.**

The check that was supposed to exist, and did not. This section previously claimed that `experiments/frozen_config.py` defines a stopping rule on the *shape* — EM stopping when $|\Delta\kappa_4| < 10^{-3}$, with a cap of 400 iterations. **That was false when written and stayed false for a month.** The config declares `em_shape_tol` and `em_max_iters`, but a search of the tree finds *no reader of either*: the rule actually implemented in `src/em.py` $\rightarrow$ `fit_em` is a relative

tolerance of 10^{-9} on the log-evidence, and the cap is whatever the caller passes. `exp_07` — the headline experiment — passed a literal 120, in four places.

This is the replicate defect of §11.2 in another costume, and it is worse in one respect: there the compendium recorded the gap once it was found, whereas here the compendium asserted a safeguard that did not exist. A document describing its own checks is evidence about those checks only if the description is itself checked.

Two mitigating facts, neither of which excuses it. The tolerance that *is* implemented is far stricter than the one advertised: replayed against the 800-update traces of `exp_27`, it fires in one of sixteen seeds, so in practice runs go to the cap rather than stopping early on a false plateau. And the direction of the exp_07 error is conservative — an under-iterated EM makes the estimator look worse, so the efficiency ratio was understated, not inflated.

The check that now exists, and what it immediately found. `exp_07` reads `FROZEN.em_max_iters`, so the budget is the configured one. `tools/check_replicates.py` is extended from replicate counts to iteration budgets, and — this is the part that matters — from settings dictionaries to *call sites*, since `n_iters=120` is a keyword argument and the dict-only version could never have seen it.

Turned on, it reports sixteen fixed budgets across three paper experiments. Sorting them by whether the run reports a coordinate the budget can actually reach:

experiment	sites	budget	reports
`exp_06`	6	80–120	ρ, q only — settle at 80, 68: adequate
`exp_06`	6	120	innovation *moments* — kurtosis settles at 229
`exp_18`	1	60	innovation moments
`exp_18`	1	60	ρ only: adequate

So the defect is not confined to `exp_07`. Eight of the sixteen budgets are defensible because the coordinates they report settle well inside them; **eight are not**, and every number downstream of those is a kernel stopped roughly half-way to its shape. None of them is quoted in the note or the workshop paper — the shape-dependent claims there come from `exp_07`, which is being rerun — but the fitted-density material in this document does depend on them, and is to be read as provisional until they are rerun at the frozen budget.

Assumption at work

An iteration budget is not a global property of a run. It is adequate or not *relative to the slowest coordinate that run reports*, and the same 120 iterations are ample for ρ and insufficient for a fourth cumulant. This is why `exp_27` exists and why its settling table is a prerequisite for reading any other experiment here, rather than a curiosity about convergence.

`experiments/exp_27_shape_convergence.py` records the whole diagnostic set at every evaluated state, which is what the original experiment never did.

11.2 The replicate counts the frozen configuration could not reach

The claim. That a frozen configuration file, imported by every experiment, was sufficient to guarantee that every number in the paper came from one protocol.

What was wrong. `frozen_config.py` exposes `n_seeds`. Four experiments carried their own replicate knobs and read none of it:

experiment	knob	was	protocol
`exp_06`	`n_rep`	10	16
`exp_06`	`n_rep_rate`	4	16
`exp_06`	`n_rep_clean`	3	16
`exp_27`	`n_seeds`	8	16
`exp_07`	`seed`	one per run, four run	16

All five runs reported `is_frozen: true`, because the provenance check compares the config against itself and cannot see a knob the experiment never asked about. *A configuration that cannot reach the parameter that matters is not freezing anything.*

The consequences were real and different in each case. The rate experiment at four replicates gives fitted log–log slopes of -0.196 for ρ and -0.700 for q against a predicted -0.500, with a non-monotone ρ curve — that is not a confirmation of the parametric rate, and the claim was pulled from the note rather than reported with a caveat. The clean-versus-noised comparison at three replicates is the same count that produced the withdrawn flatness reading of §11.3.

The check that now exists. `tools/check_replicates.py` parses each experiment's settings dictionaries and fails on any literal replicate count in a paper experiment, allowing deviations only with an explicit `# frozen-exempt: <reason>` comment. It is worth recording that this check was *first written as a shell grep*, and the grep missed `exp_27` entirely — a line-based search cannot tell a `quick` smoke dictionary from a `full` one. The AST version found it immediately. A check that is approximately right about where to look is not a check.

11.3 The flat curve that was under-replicated, not floored

The claim. On clean chains the innovation-variance error was flat in M ($0.0041, 0.0039, 0.0059, 0.0046, 0.0040$ from $M = 64$ to 1024), and the flatness was a discretisation floor imposed by the grid.

What was wrong. Both halves. At three replicates an RMSE is estimated from three numbers and its own sampling error is comparable to the effect being read off it; at sixteen the curve is not flat, running $0.0136, 0.0073, 0.0056, 0.0042, 0.0032$, close to the $M^{-1/2}$ reference. The $M = 64$ entry alone moves by a factor of three between the two replication levels.

Nor was the grid ever a plausible culprit. Removing it entirely — exact clean-data OLS on the raw transition pairs, no binning — gives $0.0135, 0.0071, 0.0057, 0.0043, 0.0032$, statistically indistinguishable from the binned fit, with a paired grid-minus-raw discrepancy of a constant 4.4×10^{-4} across all budgets. A quantity that small cannot produce a floor at 4×10^{-3}.

The lesson, which generalises. The failure mode was reading a *mechanism* into a pattern that was noise, and then reaching for the most technically interesting available explanation. The discretisation floor was a satisfying story; it was also never tested against the alternative that three points are not enough to see a trend.

11.4 Two wrong ways to compare a generated law

The claim. Successive versions compared the generated sample against clean data, and then applied a denoising read-out to it.

What was wrong. Both, in opposite directions. Reverse integration stops at $t_{\min} > 0$, so what each estimator produces is a sample of $P_{t_{\min}}$, not of P_0; comparing it against clean data measures the stopping point rather than the score. Concretely, independent Gaussian noise of variance v on an innovation of variance q and excess kurtosis κ gives excess kurtosis $\kappa(q/(q+v))^2$, since fourth cumulants add and the variance grows. The second version applied a denoising read-out instead, which inverted the bias through shrinkage.

What replaced it. Draw the reference from P_0 and noise it *forward* to the same $t_{\min}$, so both sides are samples of the same law and the target follows in closed form. At $\rho = 0.85$, $t_{\min} = 0.02$ the residual innovation noise is $v = \Delta_t(1 + \rho^2)$ and the predicted target is 1.9098 against a clean 3.0. The closed-form target is what identified both errors: neither was visible as an anomaly, and both produced plausible numbers.

11.5 A sign error in a gradient that a fixed point could not see

What was wrong. `src/kernels.py` → `MixtureInnovationKernel.grad_log_transition_matrix` carried a sign error in the ρ derivative.

Why it mattered more than it looks. A fixed-point residual near zero is evidence of a stationary point only if the derivative defining it is right. Every convergence diagnostic downstream of that function inherited the error, so the runs looked converged for a reason unrelated to whether they were.

The check that now exists. Fisher's identity is verified against finite differences of the discretised evidence to $\sim 10^{-9}$ — a check that compares two independent routes to the same quantity, rather than one that asks a quantity whether it is happy with itself.

11.6 The ring normaliser, twice

Recorded in full in §10.4 and its surrounding boxes; summarised here because it is the most recent and the most instructive.

The ring density is written unnormalised, and its normaliser depends on the parameters of the confining well. Omitting it is harmless while only ψ is being estimated — the term is constant and cancels in every responsibility — and silently fatal the moment λ is estimated, because a wider well contains more mass and the likelihood grows without bound.

Two things are worth extracting. First, the bug was *introduced by a correct-looking port*: the ported code agreed with its audited original bit for bit, because the original only ever compared ψ at fixed λ and never needed the term. Agreement with a reference only certifies the cases the reference was used for. Second, fixing it in `src/ring.py` → `log_pt_conditional` left it unfixed in `src/ring.py` → `log_pt_marginal`, where the count differs — once per frame rather than once — and that second instance was caught not by a test but by an experiment diverging to $\hat{\lambda} = 3.5 \times 10^6$. The test suite now pins both functions and both directions.

11.7 What these have in common

Five of the six were failures of *measurement*, not of derivation. No theorem was wrong. In each case a quantity was computed correctly and then read as evidence for something it could not support: an iteration budget read as a capacity limit, three replicates read as a floor, a stopping point read as a score error, a fixed point read as convergence, a constant read as absent.

Two habits follow, and they are the reason this project's checks look the way they do.

Prefer two independent routes to one self-consistent one. The gradient sign error survived every internal diagnostic and died immediately against finite differences. The generation comparison survived two revisions and died against a closed-form target. Grid BP is checked against a closed form *and* against brute-force enumeration sharing no code with it.

Make the failing direction fail. It is not enough for a test to pass when the code is right; where a bug's signature is known, assert it. `tests/test_ring.py` asserts that the profile likelihood peaks at the truth *and* that it becomes monotone when the normaliser is removed, so deleting the term cannot make the suite greener. Most of the entries above would have been caught a good deal sooner by a test of that shape.

Bibliography

[1] Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

[2] Ulrich G. Haussmann and Etienne Pardoux. Time reversal of diffusions. *The Annals of Probability*, 14(4):1188–1205, 1986.

[3] Herbert Robbins. An empirical Bayes approach to statistics. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability, Volume 1*, pages 157–163, 1956.

[4] Koichi Miyasawa. An empirical Bayes estimator of the mean of a normal population. *Bulletin of the International Statistical Institute*, 38(4):181–188, 1961.

[5] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[6] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[7] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

[8] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, 2019.

[9] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

[10] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

[11] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, 2009.

[12] Martin J. Wainwright and Michael I. Jordan. Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2):1–305, 2008.

[13] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1):164–171, 1970.

[14] Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[15] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[16] Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[17] Olivier Cappé, Eric Moulines, and Tobias Rydén. *Inference in Hidden Markov Models*. Springer, 2005.

[18] Song Mei. U-Nets as belief propagation: efficient classification, denoising, and diffusion in generative hierarchical models, 2024.

[19] Steven M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993.

[20] Erich L. Lehmann and George Casella. *Theory of Point Estimation*. Springer, 2nd edition, 1998.

[21] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38, 1977.

[22] C. F. Jeff Wu. On the convergence properties of the EM algorithm. *The Annals of Statistics*, 11(1):95–103, 1983.

[23] Xiao-Li Meng and Donald B. Rubin. Maximum likelihood estimation via the ECM algorithm: a general framework. *Biometrika*, 80(2):267–278, 1993.

[24] Ronald A. Fisher. Theory of statistical estimation. *Mathematical Proceedings of the Cambridge Philosophical Society*, 22(5):700–725, 1925.

[25] Thomas A. Louis. Finding the observed information matrix when using the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 44(2):226–233, 1982.

[26] Henry Teicher. Identifiability of finite mixtures. *The Annals of Mathematical Statistics*, 34(4):1265–1269, 1963.

[27] Sidney J. Yakowitz and John D. Spragins. On the identifiability of finite mixtures. *The Annals of Mathematical Statistics*, 39(1):209–214, 1968.

[28] Jérôme Garnier-Brun, Marc Mézard, Emanuele Moscato, and Luca Saglietti. How transformers learn structured data: insights from hierarchical filtering, 2024.